Proof-Valid Serving and Benchmarking for Approximate LLM Caches under Premise Erasure

Jianfeng Xu[1*]

[1*]Koguan School of Law, China Institute for Smart Justice, School of Computer Science, Shanghai Jiao Tong University, Shanghai, 200030, China.

Corresponding author(s). E-mail(s): xujf@sjtu.edu.cn;

Abstract

Semantic similarity tells an approximate LLM cache whether a stored response matches a request, not whether the evidence behind that response is still available and eligible. We present PVCacheServe, a contract-based framework that binds each cached response to declared support records and gates reuse on an available certificate at a named audit state. Misses and failed checks enter fallback regeneration, accepted repairs are written back with fresh records, and protected payloads repair unreadable evidence only after eligibility is rechecked, because restored bytes cannot reinstate superseded evidence. Building on a residual-leaf law from companion theory, we characterize fixed-state availability, extend it to fallible backups, and derive selective-regeneration identities separating cached errors avoided from fallback errors introduced. On a frozen-cache daily Wikipedia replay with two generators, an asserted sentence contract intercepts **98.1%** of source-defined stale cases at **18.6**–**19.7%** fallback demand, versus **95%** for one-day TTL. Human audits show that lexical construction and fallback acceptance, not staleness, dominate the projected error budget: higher interception alone does not reduce erroneous delivery. A monthly Wikipedia/Wikidata replay executes the full lifecycle: the asserted-claims loop needs **29** accepted regenerations versus **421** for blind refresh, delivers no annotated-wrong endpoint answer under either generator, and halves read-loss regenerations without trusting ineligible copies. Dependency scope and fallback quality, not similarity, are the levers of evidence-aware cache maintenance.

Keywords: approximate LLM caching, evidence contracts, certificate gating, knowledge freshness, premise erasure, selective regeneration

1 Introduction

Similarity does not establish evidence currency.

Approximate large-language-model (LLM) caches reduce repeated inference by reusing responses for semantically similar requests [1–4]. A similarity hit, however, does not establish whether the evidence recorded for the response remains available or eligible. Matching relates a request to a cached entry; deletion, supersession, and access restrictions concern the entry's dependencies.

We use *premise erasure* for the loss or invalidation of an evidence unit required by a declared reuse contract. Two cases require different remedies. A protected copy may repair an unreadable payload, but recovering old bytes cannot restore the eligibility of superseded or withdrawn evidence. Evidence-aware reuse must therefore distinguish payload recovery from current eligibility.

The maintenance challenge is to reject entries whose required dependencies no longer hold without refreshing entries merely because unrelated source content changed. For example, a Wikipedia lead sentence may persist after an infobox value changes, or be rewritten while that value remains unchanged. Sentence persistence, structured-value equality, page versioning, and time-to-live expiry consequently implement different reuse contracts rather than interchangeable freshness tests. The question is not only which entries a policy rejects, but whether replacing their cached answers improves delivery quality.

Contract-based reuse and selective maintenance.

We introduce *proof-valid cache serving and benchmarking* (PVCacheServe), a framework that separates candidate matching, declared-evidence availability, and answer correctness. A cache entry stores a response-conditioned support schema and premise records. Figure 1 shows the reuse decision at a fixed audit state: after matching proposes a response, the gate checks whether an accepted schema for the current request–response pair admits an available certificate. A failed check or cache miss enters fallback; an accepted repair receives fresh support records and may be written back.

Here *proof-valid* means valid under the accepted evidence contract and checked state, not independently established request compatibility, semantic sufficiency, or factual correctness.

The evaluation therefore asks two separate questions: which entries a contract rejects, and whether replacing them improves what users receive.

Contributions.

- **Contract semantics for evidence-dependent reuse.** We separate candidate matching, declared-support availability, and answer correctness, and specify when a restored payload may re-enter checking: integrity is necessary but never sufficient, because recovered bytes cannot restore superseded or withdrawn eligibility.
- **Availability and error accounting.** We restate the residual-leaf law of the companion theory [5] for response-conditioned contracts with eligibility, extend

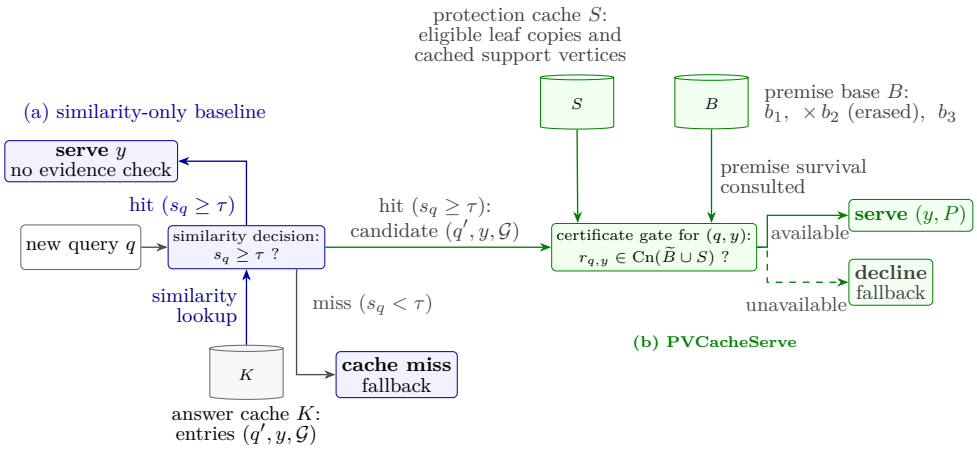


Fig. 1 Fixed-state reuse decision. A similarity-only baseline serves a threshold-accepted candidate directly. PVCacheServe additionally requires an accepted support schema for the current pair (q, y) and an available certificate within that schema. The answer cache K supplies candidate responses and support records; the protection cache S supplies directly available eligible support objects. All checks refer to one audit state. Maintenance after a miss or failed check is shown in Figure 2.

it to fallible backups, and derive selective-regeneration identities that separate cached errors avoided from fallback errors introduced. On shared fallback outputs, the rejected-set form gives an exact, label-based improvement criterion.

- **An executed lifecycle with separated outcome measures.** A dual-generator daily replay measures how dependency scope trades stale delivery against fallback demand; human audits show that construction and fallback acceptance, not staleness, dominate the projected error budget. A monthly replay executes fallback, write-back, and eligibility-checked restoration with human-labeled endpoint quality.

Principal findings.

(i) Dependency scope sets maintenance cost: narrowing the monthly contract from all designated evidence to answer-asserted claims reduces confirmed-correct declines from 38 to 18 of 118 and multi-tick regenerations from 53 to 29, while still declining every decayed hit. (ii) Interception is the wrong objective: a daily contract that intercepts 98.1% of stale cases still projects more erroneous deliveries than similarity-only reuse, because set-level fallback acceptance admits wrong-part answers; per-part acceptance reverses the point projection. (iii) Recovery and eligibility must be separated: trusting restored base-period bytes re-serves all eight decayed answers, whereas eligibility-checked restoration halves read-loss regenerations.

Organization.

Section 2 positions related work. Sections 3–5 develop the contracts, analysis, and workflow. Sections 6–7 present the evaluation and results. Section 8 discusses implications and limitations, and Section 9 concludes. Appendices A–D provide analytical details, data and annotation protocols, execution procedures, and an artifact guide.

Online Resource 1 supplies extended audits, case analyses, sensitivity analyses, and file-level artifact mappings.

2 Related Work and Positioning

Semantic matching and inference-state reuse.

GPTCache uses semantic matching for response reuse [1]; vCache learns prompt-specific thresholds with error guarantees under its stated assumptions [2, 6]. MeanCache learns query-similarity models for user-side caching [4], while Krites judges candidate responses and promotes approved matches into a dynamic cache [3]. These mechanisms address whether a cached response is suitable for a request. PVCacheServe adds a separate check of its declared dependencies. Inference-state caching and serving optimizations operate at a different layer [7–11]. This work concerns evidence-dependent answer reuse.

Evidence evaluation and temporal knowledge.

RAGAs, ARES, and RAGChecker assess retrieval and answer quality, including support from supplied evidence [12–14]. Data provenance records dependencies [15, 16], while corpus-mutation testing studies responses to knowledge changes [17]. TemporalWiki, WikiFactDiff, RealTimeQA, and FreshLLMs study evolving knowledge and answer freshness [18–21]. DailyQA constructs questions from daily Wikidata changes [22]. Our daily replay adopts this change-centered construction idea but uses templated three-part questions and recorded support sentences to compare dependency scopes. It evaluates cache routing and fallback consequences rather than a model's temporal knowledge alone. Evidence availability remains distinct from entailment and answer correctness.

Dependency-aware answer reuse.

GroundedCache combines semantic matching with evidence overlap, version validity, and support checks [23]. FinCacheServe uses intent, document-version, evidence, tool, model, and decoding guards [24]. FreshCache estimates tier-specific staleness risk and distinguishes detected content changes from changes affecting answer correctness [25]. PVCacheServe's novelty lies not in the validation predicate—an exact validator with the same dependencies, state, and eligibility rule makes identical flat-conjunctive decisions—but in the contract that links this predicate to failure exposure, eligibility-preserving restoration, and selective-regeneration accounting, and in an evaluation that separates routing from delivered quality. We evaluate overlap and equality mappings motivated by GroundedCache and FinCacheServe over our evidence representation, with page hashing as a generic change-detection reference; none of these reproduces the original systems or FreshCache's tiered predictor.

Companion theory and protection references.

Coded caching studies storage–delivery trade-offs [26, 27], and erasure codes provide payload-recovery constructions [28–31]. The companion manuscript [5] studies proof-valid caching of explicit logical consequences under independent premise erasure, with

the query itself excluded from the cache. It proves a residual-leaf law, reduces cache design to partial path interception (PPI), shows PPI NP-complete on unit-cost depth-two DAGs and W[1]-hard in the budget, gives exact algorithms for unique-path and bounded-treewidth instances, characterizes the shared-workload optimum under exact module routing, and compares transparent storage with a coded payload-recovery benchmark.

3 Evidence Contracts and Audit Semantics

3.1 Premise availability and eligibility

A reuse contract declares the evidence units required by a cached response and the conditions under which they remain usable. We represent each unit as a *premise leaf*. Its granularity—for example, a sentence, chunk, or document—defines the unit of erasure.

Definition 3.1 (Premise-leaf record)**.** A premise leaf b carries

$$b = (\mathrm{id}_b, h_b, v_b, \mathsf{ix}_b, \mathsf{pol}_b, \pi_b), \tag{3.1}$$

where the fields specify source identity, content hash, version, retrieval lineage, policy tag, and payload. At audit state t, its availability is

$$\begin{aligned}\mathrm{Avail}(b,t) = \mathrm{Present}(b,t) \wedge \mathrm{HashMatch}(b,t) \\ \wedge \mathrm{VersionCompatible}(b,t) \wedge \mathrm{PolicyAllowed}(b,t) \wedge \mathrm{IndexReachable}(b,t).\end{aligned} \tag{3.2}$$

The predicates concern payload access and integrity, current eligibility, and the access route required by the contract.

Premise identity includes the version and eligibility distinctions required by the contract. Repeated references to one identity denote one evidence unit, not independent copies. Within a schema they use one leaf; under a shared source-loss realization they use one survival indicator. Occurrence-local failures are used only when explicitly declared.

Deletion, supersession, failed reads, and retrieval changes can affect different predicates in Eq. (3.2). Their effects need not be independent. Omission from a top-k result counts as unavailability only when that retrieval window is part of the contract.

Payload recovery does not restore eligibility.

A protected copy can repair an unreadable payload only while the recorded content remains eligible. Recovering old bytes cannot reverse supersession or permission withdrawal. A permitted backup route need not reproduce the failed source route, but its payload must still satisfy the contract's integrity, eligibility, and access conditions.

Below, $\widetilde{B}$ denotes premises available through source routes, and S contains eligible objects available through protection routes. The analytical laws fix support, eligibility, and S, and randomize source-premise erasures. The executed fault experiments additionally permit backup-read failures and recheck eligibility at each snapshot.

3.2 Support schemas and available certificates

Certificate checking evaluates a stored response under a declared support contract. It does not model the language model as a deduction engine or independently establish that the declared evidence entails the response.

Definition 3.2 (Response-conditioned support schema)**.** For a request q and response y, a support schema is a finite DAG $\mathcal{G}_{q,y} = (V_{q,y}, A_{q,y})$ with a distinguished internal service sink $r_{q,y}$. Premise vertices have no parents and carry premise records. Every internal vertex has at least one parent and represents a declared support assertion or reusable support object. Edges point from prerequisites to dependents. The sink represents certification of y for q under the contract. For a fixed stored response y_q, write $G_q = \mathcal{G}_{q,y_q}$ and abbreviate its sink by q.

Let $\mathrm{pa}(v)$ be the parents of vertex v. Shared premise identifiers denote the same evidence unit. Shared internal identifiers denote the same support object and dependency rule; service sinks remain request-specific.

Assumption 3.1 (Deterministic certificate semantics)**.** A premise leaf is available exactly when it survives through its source route or has a directly available eligible copy. An internal vertex is available exactly when it is directly available from the eligible protection cache or all its parents are available. Transparent DAG closure does not reconstruct erased leaves. Coded recovery uses a separate decoder, whose outputs enter checking only after the remaining eligibility conditions pass.

Let B be the premise universe, $\widetilde{B} \subseteq B$ the source-available premises, and S the directly available eligible protection objects. The set S excludes service sinks: storing a response in the answer cache does not make its certification predicate available.

For any set X of vertex identities, let $\mathrm{Cn}_q(X)$ be the least set $C \subseteq V(G_q)$ satisfying

$$X \cap V(G_q) \subseteq C, \qquad \mathrm{pa}(v) \subseteq C \Longrightarrow v \in C \quad \text{for each internal } v.$$

Leaves enter only through X. This closure is monotone; write Cn when the schema is clear.

Definition 3.3 (Proof-valid serving)**.** For fixed q and y_q, an available certificate is a set P such that

$$\mathrm{Cert}_{q,y_q}(P; \widetilde{B}, S) = 1 \quad \Longleftrightarrow \quad P \subseteq (\widetilde{B} \cup S) \cap V(G_q) \text{ and } q \in \mathrm{Cn}_q(P).$$

Serving is proof-valid under the contract when such a certificate is returned or made accessible for verification. Otherwise reuse is declined.

A certificate need not be minimal. A cached internal object bypasses its prerequisites only when the contract permits that substitution. Identifiers and hashes bind records but do not replace any payload required by the contract.

Lemma 3.1 (Certificate-existence criterion)**.** *For fixed G_q and audit state $(\widetilde{B}, S)$, an available certificate exists if and only if $q \in \mathrm{Cn}_q(\widetilde{B} \cup S)$.*

Proof An available certificate P is a subset of $(\widetilde{B} \cup S) \cap V(G_q)$, so monotonicity gives the forward implication. Conversely, if the closure contains q, then $P = (\widetilde{B} \cup S) \cap V(G_q)$ is an available certificate. □

All checks refer to one audit state. A deployment must keep that state applicable through delivery, for example using a consistent snapshot or validated state token; the model does not resolve concurrent-update races.

If y was cached for another request q', reuse requires an accepted schema for the current pair (q, y). Availability under $\mathcal{G}_{q',y}$ alone does not establish compatibility with q. The experiments do not provide a general method for transferring support contracts between requests.

3.3 Constructing accepted support

Definition 3.4 (Certificate-construction interface)**.** A constructor attempts to produce a support schema from a request, response, corpus state, and observation log:

$$\Pi : (q, y, \mathcal{D}_t, \mathcal{L}_t) \longmapsto \mathcal{G}_{q,y} \text{ or } \bot. \tag{3.3}$$

A returned schema must be accepted under the evidence contract before use. Its evidence source is one of:

- *Instrumented provenance*: records collected during generation or retrieval, including source identities and hashes;
- *Benchmark annotation*: designated support supplied by the dataset, not necessarily used by a particular generation;
- *Post-hoc reconstruction*: support selected from (q, y) and corpus access, without establishing fidelity to an unobserved generation log.

Assumption 3.2 (Constructible support)**.** Every pair admitted to proof-valid evaluation has an explicit contract and either an accepted conjunctive schema satisfying Assumption 3.1 or a finite family of such schemas as defined in Appendix A.2. Without an accepted representation, reuse is declined.

3.4 Matching and contract checking

Matching proposes a cached candidate with score $s_q \geq \tau$; contract checking evaluates its accepted support representation. For class-aware benchmark accounting, let $C(q)$ be the declared reuse class. A *false hit* serves a candidate outside $C(q)$. A *premise error* serves an admissible candidate without an available certificate. Foreign-class serves count only as false hits, making the categories disjoint.

An exact gate excludes premise errors relative to the contract and checked state. It does not establish class admissibility or answer correctness, and similarity alone does not establish evidence availability. Supplementary Section S1 reports the corresponding synthetic audits.

Conjunctive and majority contracts.

The strict, claims-only, and asserted-claims implementations are flat conjunctions. A majority contract requires all designated claims and, for $m > 0$ designated sentences, at least $\lfloor m/2 \rfloor + 1$ sentences. It is therefore a family of conjunctive certificates containing all required claims and each sentence subset of that size. For $m = 0$, the sentence condition is true. The residual law applies within each member, and Appendix A.2 treats family availability. Read-protection experiments use the strict and asserted-claims contracts.

4 Availability and Conditional Protection

Fix a request q, its stored response y_q, a conjunctive schema G_q, and an eligible protection cache S. Only source-premise survival is random; the support rules, eligibility conditions, and cached objects remain fixed. Lemma 4.1 and Theorem 4.1 restate the companion's failure-path characterization and residual-leaf law [5]. Two changes require self-contained proofs: S contains eligible protection objects rather than derivable consequences, and Assumption 3.1 replaces derivation closure with declared support rules. Excluding the service sink from S plays the role of the companion's ban on caching the query.

Definition 4.1 (Exposed premises)**.** Let $V_0(G_q)$ be the premise leaves and $U(q, S) = S \cap V(G_q)$. A directed path to the service sink is *cache-free* if it contains no vertex of $U(q, S)$. Define

$$\begin{aligned} D_{\exp}(q, S) &= \{b \in V_0(G_q) : \text{a cache-free path from } b \text{ to } q \text{ exists}\}, \\ e_q(S) &= |D_{\exp}(q, S)|. \end{aligned} \tag{4.1}$$

Thus a premise remains exposed if any path from it to the sink avoids protection. A reverse traversal after deleting $U(q, S)$ computes the exposed leaves in $O(|V(G_q)| + |A(G_q)|)$ time.

Lemma 4.1 (Residual failure path; cf. [5])**.** *Under Assumption 3.1,*

$$q \notin \mathrm{Cn}_q(\widetilde{B} \cup S) \iff D_{\exp}(q, S) \not\subseteq \widetilde{B}. \tag{4.2}$$

Proof If an erased exposed leaf has a cache-free path to q, the leaf is unavailable and each successive uncached vertex has an unavailable parent. Hence q is unavailable. Conversely, an unavailable internal vertex is uncached and has an unavailable parent. Tracing such parents backward in the finite DAG reaches an erased uncached leaf, producing a cache-free path to q. □

Theorem 4.1 (Residual-premise law; cf. [5])**.** *Under Assumption 3.1, suppose distinct source premises are erased independently with probability* $0 \le \varepsilon < 1$*. For fixed* G_q *and eligible* S*,*

$$\Pr_{\varepsilon}\Big[q \in \mathrm{Cn}_q(\widetilde{B} \cup S)\Big] = (1-\varepsilon)^{e_q(S)}. \tag{4.3}$$

For $0 < \varepsilon < 1$ *and* $0 < \delta < 1$*, reliability at least* $1-\delta$ *is equivalent to* $e_q(S) \le N^*(\varepsilon, \delta)$*, where*

$$\begin{aligned} N^*(\varepsilon,\delta) &= \max\{n \in \mathbb{Z}_{\ge 0} : (1-\varepsilon)^n \ge 1-\delta\} \\ &= \left\lfloor \frac{\log(1-\delta)}{\log(1-\varepsilon)} \right\rfloor. \end{aligned} \tag{4.4}$$

Set $N^*(0,\delta) = +\infty$*.*

Proof Lemma 4.1 makes certification equivalent to survival of every leaf in the fixed set $D_{\exp}(q,S)$. Independence gives Eq. (4.3). For $0 < \varepsilon < 1$, solving $(1-\varepsilon)^{e_q(S)} \ge 1-\delta$ gives Eq. (4.4). At $\varepsilon = 0$, every finite exposed set survives. □

The deterministic equivalence does not require independence. Under a dependent failure law, availability is instead

$$\Pr[D_{\exp}(q,S) \subseteq \widetilde{B}],$$

which is not determined by marginal erasure probabilities alone. Section 7.1 evaluates this distinction. Appendix A.1 bounds sensitivity to exposed-count misspecification.

4.1 Transparent leaf protection

For flat conjunctive contracts, protection reduces the number of required source reads. Write $a^+ = \max\{a, 0\}$, and let $c_B > 0$ be the cost of one equal-sized premise packet.

Corollary 4.1 (Leaf-only transparent baseline)**.** *Fix an eligible conjunctive schema whose premise leaves reaching the service sink are exactly the* κ_q *distinct packets in* P_q^**. Under independent source erasures with* $0 < \varepsilon < 1$*, restrict protection to reliable transparent copies of these packets. For* $0 < \delta < 1$*, the minimum storage achieving reliability* $1-\delta$ *is*

$$\sigma^*_{\mathrm{leaf}}(q,\varepsilon,\delta) = \big(\kappa_q - N^*(\varepsilon,\delta)\big)^+ c_B. \tag{4.5}$$

At zero erasure, no protection is required.

Proof Protecting x distinct required leaves leaves $\kappa_q - x$ exposed. Theorem 4.1 requires $\kappa_q - x \le N^*(\varepsilon,\delta)$. □

The executed store permits backup reads to fail. For a fixed eligible flat contract, the corresponding law is:

Corollary 4.2 (Fallible transparent leaf protection)**.** *Let P_q be the distinct required premises of an eligible flat conjunctive schema, and let T identify premises with fingerprint-matching backup copies. Source reads fail with probability ε, and backup reads fail with probability ε'. If all reads are independent, then*

$$\Pr[q \text{ is certified}] = (1-\varepsilon)^{|P_q \setminus T|}(1-\varepsilon\varepsilon')^{|P_q \cap T|}. \tag{4.6}$$

For $\varepsilon' = 0$, this is the residual law with $S = P_q \cap T$.

Proof An unprotected premise is readable with probability $1-\varepsilon$. A protected premise is unreadable only if both routes fail, with probability $\varepsilon\varepsilon'$. Certification requires every distinct premise to be readable, giving the product. □

An ineligible required premise causes rejection regardless of protection. These formulas describe individual checks at a fixed state; write-back changes the required sets and therefore requires trajectory execution.

Alternative certificates are handled separately in Appendix A.2: each conjunctive branch has its own exposed set, and the family succeeds when any accepted branch certifies the response. Supplementary Sections S2.3 and S5 evaluate constructed families without establishing their semantic sufficiency.

4.2 Placement implications and coded reference

For fixed eligible flat contracts, reliable protected copies, weights $w_q \geq 0$, and premise costs $c_b > 0$, the residual law gives the workload objective

$$\max_{T:\, \sum_{b\in T} c_b \leq B_{\text{prot}}} \sum_q w_q (1-\varepsilon)^{|P_q \setminus T|}.$$

For $0 < \varepsilon < 1$, adding an unprotected premise b yields the marginal gain

$$\varepsilon \sum_{q:\, b \in P_q \setminus T} w_q (1-\varepsilon)^{|P_q \setminus T|-1}.$$

The gain depends on residual footprint as well as coverage, and allocation must also account for payload cost. Coverage frequency alone therefore does not optimize this objective.

Two facts motivate a heuristic. Flat contracts are unique-path schemas, for which the companion gives exact per-certificate algorithms [5]; here the per-certificate optimum is immediate (Corollary 4.1). The shared byte-budgeted objective is hard regardless of schema depth: with singleton contracts $P_q = \{b_q\}$ it equals $\sum_q w_q(1-\varepsilon) + \varepsilon \sum_{q:\, b_q \in T} w_q$, so maximizing it under heterogeneous costs c_b contains the 0–1

knapsack problem. The executed placement (Section 5.4) is therefore a coverage-ranked greedy heuristic, and N^* remains a per-certificate reference. Supplementary Section S6 evaluates the retained placements.

Coded payload recovery.

Appendix A.3 gives a fixed-split systematic-MDS reference for designated payload recovery. Within that ideal equal-cost family, pure parity weakly dominates transparent–parity splits (Remark A.1). Decoded payloads still require eligibility checks. No encoder–decoder is executed; Supplementary Section S2 reports the numerical references.

5 An Executable Certificate-Carrying Reuse Workflow

Figure 2 extends the fixed-state decision in Figure 1 to maintenance across update ticks, including fallback, write-back, and protected read repair. Algorithm 1 specifies the validation obligations; Section 5.3 identifies the replay's implementation boundary. The prototype uses flat evidence records, while the formal checker also admits finite families of accepted conjunctive schemas.

5.1 Stores and audit state

The prototype maintains an answer cache with retrieval aliases, a support-record store, source and protection payload stores, and an eligibility registry. Cache entries have the form $(q', y, \mathcal{G})$: a cached request, its response, and its support records. Support records bind each response to its designated claims, lexically extracted asserted claims, sentence identities, and issuance state. Payload stores retain serialized claim-value keys and sentence text with fingerprints. The registry separately records context-specific eligibility and the last eligible version.

Claim identities use *entity* | *property* | *value*; sentence identities use *page* | *sentence-id*. A state provider materializes these records from the snapshots. Issuance and verification states are named separately.

The physical protection store retains candidate payloads. Only readable, integrity-checked objects that remain eligible at the verification state contribute to the available protection set S of Section 3. Retaining a payload or support record does not establish its current availability.

5.2 Contract checking

Missing or unreadable evidence is unavailable. A conjunction fails if any required item is unavailable; a family succeeds if any accepted branch certifies the response. Matching and support acceptance remain separate from this availability check: the matching stage proposes candidates, and the gate evaluates the certificate predicate of Definition 3.3 without adjudicating candidate admissibility (Section 3.4).

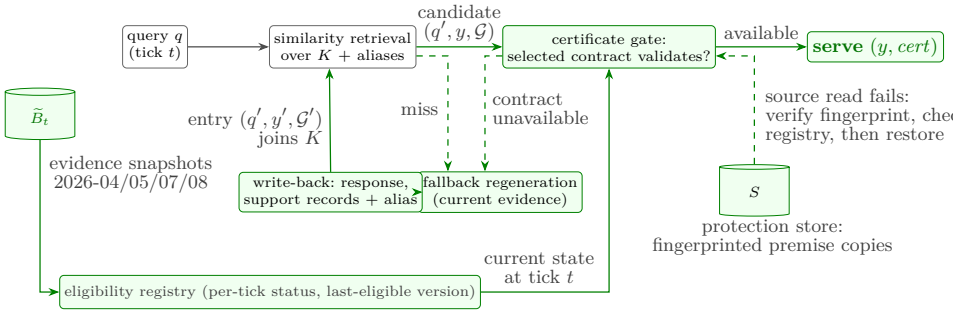


Fig. 2 Maintenance lifecycle across update ticks. Misses and failed contract checks enter fallback using current evidence. Accepted repairs may be written back with response-bound support records and a permitted retrieval alias, changing the cache state for later requests. Protected payloads repair failed reads only after integrity and current-eligibility checks. Absent usable evidence or failed acceptance produces abstention.

Proposition 5.1 (Correctness of exact family checking)**.** *Fix a request–response pair* (q, y)*, an audit state* $(\widetilde{B}_t, S)$*, and a finite nonempty family of accepted conjunctive schemas* $\mathcal{F} = \{G_{q,j} : j \in J_q\}$*. Evaluate each schema separately, without combining its dependency rules with those of other schemas. If the checker accepts exactly when*

$$\bigvee_{j \in J_q} \mathbf{1}\Big[q \in \mathrm{Cn}_{q,j}(\widetilde{B}_t \cup S)\Big] = 1,$$

then acceptance admits an available certificate in $\mathcal{F}$ (soundness)*, and rejection implies that no such certificate exists* (family-completeness)*.*

Proof Apply Lemma 3.1 to each schema and take the disjunction of the resulting certificate-existence predicates. □

Fail-closed checking alone does not imply completeness. Timeouts, omitted branches, or conservatively unresolved predicates can decline certifiable candidates. The proposition assumes exact checking at the named state; it does not establish state-provider accuracy, semantic sufficiency of the accepted schemas, or validity across concurrent state changes.

5.3 Fallback and write-back

The monthly implementation builds fallback evidence from the current tick: the property's current values admitted by the claim projection and up to three current-page sentences mentioning them. Generation follows the base-cache model configuration and lexical value-mention acceptance rule. Absent usable evidence or unsuccessful acceptance produces abstention.

An accepted output receives records for the current claims and sentences. Its asserted set contains lexically matched values, not an exhaustive semantic decomposition. The replay constructs these records at the issuance snapshot from values

Algorithm 1 Contract-gated reuse with selective repair: reference procedure

Require: request q, audit state t, answer cache K, protection store, selected contract, write-back setting

retrieve a candidate response y and its support records
if matching accepts the candidate and its support representation is accepted for (q, y) **then**
  check required evidence at t, attempting permitted restoration of protected read losses
  if an available certificate P validates **then**
    return cached response y and accessible P
  end if
end if
obtain evidence permitted at t
if no usable evidence is available **then**
  return abstention
end if
generate a response y' from that evidence
if generation acceptance succeeds **then**
  construct support records bound to y'
  if the representation is accepted for (q, y') and an available certificate P' validates at t **then**
    if write-back is enabled, install the response, support records, and permitted retrieval alias
    return regenerated response y' and accessible P'
  end if
end if
return abstention

admitted by the current claim projection and from current-page sentences. Every recorded premise is therefore available at issuance by construction, which discharges the issuance check of Algorithm 1; the replay does not repeat it as a separate call. Aliases support later matching but do not establish request equivalence.

The replay regenerates from the cached question rather than the incoming paraphrase. Its scope regarding query drift is specified in Section 6.3. Archived outputs permit replay without new model calls under the coverage protocol of Appendix C.1.

5.4 Budgeted payload restoration

A failed source read of a protected premise triggers a backup read, fingerprint verification, and current-eligibility checking. Only a readable, matching, eligible payload may re-enter the contract check. Restoration cannot reverse supersession or permission withdrawal: restored bytes re-enter only as candidates, never as eligibility (Section 3.1).

Placement uses base-period support records only. Payload objects are ranked by certificate-reference frequency, with ties broken by ascending storage key. A greedy

scan stores each payload that fits the remaining byte budget and skips those that do not. Placement remains fixed across update ticks. Write-back items with payload keys outside the base placement universe are unprotected; current eligibility is checked in the item's request-specific context. The exposed-set accounting of Definition 4.1 therefore reflects the actual placement along the trajectory.

Appendix C specifies byte accounting and fault coupling; Supplementary Section S6 reports static placement ablations.

5.5 When selective regeneration reduces error

Contract-valid reuse does not guarantee correct delivery. The following model separates stale cached delivery, other cached errors, fallback errors, and abstention.

Proposition 5.2 (Selective-regeneration operating window)**.** *Consider N cases with fixed staleness labels, of which s are stale. A policy rejects $\rho > 0$ cases, including ρ_s stale cases, and reuses the rest. Thus $m = s - \rho_s$ stale cases are cache-served. For every rejected case, fallback delivers with probability a and has conditional error probability r. Every non-stale cached case has error probability b. These probabilities are common across cases and compared policies. Errors are wrong or incomplete delivered answers, and every stale cached delivery is an error. Let E be expected erroneous deliveries and $\pi = \rho_s/\rho$ the policy's rejection precision. Then:*

(i)

$$E_{\text{pol}} - E_{\text{sim}} = \rho(ar - b) - \rho_s(1 - b) = \rho\big[ar - \pi - (1 - \pi)b\big].$$

For $b < 1$, improvement over similarity-only reuse is equivalent to

$$\frac{ar - b}{1 - b} < \pi. \tag{5.1}$$

(ii) For a reference with $\rho' > \rho$ rejections and no stale cached delivery,

$$E_{\text{pol}} - E_{\text{ref}} = m(1 - b) - (\rho' - \rho)(ar - b).$$

The policy improves on that reference exactly when $ar - b > m(1 - b)/(\rho' - \rho)$.

(iii) Expected abstentions equal $\rho(1 - a)$.

Proof The expected counts are

$$\begin{aligned} E_{\text{pol}} &= m + (N - \rho - m)b + \rho ar, \\ E_{\text{sim}} &= s + (N - s)b, \\ E_{\text{ref}} &= (N - \rho')b + \rho' ar. \end{aligned}$$

Subtracting gives (i) and (ii); (iii) follows from fallback acceptance. No independence across cases is required. □

When $\rho_s > 0$, precision is the reciprocal of fallback invocations per intercepted stale case. Higher precision permits greater normalized excess fallback error before

selective regeneration becomes worse than reuse. The sign of $ar - b$ determines the error-optimal rejection regime.

Corollary 5.1 (Error-optimal rejection regimes)**.** *Under Proposition 5.2, consider all feasible rejection sets on the same stream, including $\rho = 0$.*

(i) If $ar > b$, then

$$E_{\text{pol}} \geq E_{\text{sim}} - s(1 - ar),$$

with equality when exactly the stale cases are rejected. Such a policy is error-optimal. At fixed interception ρ_s, error increases with ρ.

(ii) If $ar < b$, then $E_{\text{pol}} \geq N\,ar$, with equality when every case is rejected. Any nonempty rejection set improves on similarity-only reuse, and error decreases with ρ at fixed ρ_s.

(iii) If $ar = b$, then

$$E_{\text{pol}} = E_{\text{sim}} - \rho_s(1 - b),$$

so every policy with no stale cached delivery is error-optimal.

Proof Use $E_{\text{pol}} = E_{\text{sim}} + \rho(ar - b) - \rho_s(1 - b)$. For (i), $\rho \geq \rho_s$ gives $E_{\text{pol}} \geq E_{\text{sim}} - \rho_s(1 - ar)$; then use $\rho_s \leq s$. For (ii), $\rho \leq N$ and $\rho_s \leq s$ give the lower bound, attained by rejecting every case. Both terms in the difference from similarity-only reuse are nonpositive, the first strictly negative for $\rho > 0$. Part (iii) is immediate. □

Remark 5.1 (Rejected-set form of the window). For a one-shot comparison with shared cached answers and one shared fallback output per case, neither homogeneous quality nor stale-is-error is needed. Let c_i indicate erroneous cached delivery and f_i erroneous fallback delivery, with $f_i = 0$ on abstention. If the policy's fallback set R contains the baseline set R_{sim}, then

$$\text{Err}_{\text{pol}} - \text{Err}_{\text{sim}} = \sum_{i \in \Delta R} (f_i - c_i), \qquad \Delta R = R \setminus R_{\text{sim}}.$$

For nonempty ΔR, improvement is equivalent to $\bar{f}_{\Delta R} < \bar{c}_{\Delta R}$. Under Proposition 5.2 with $R_{\text{sim}} = \emptyset$, their expectations are ar and $\pi + (1 - \pi)b$, recovering Eq. (5.1). Write-back generally prevents this comparison because later cached answers become policy-dependent.

These results compare reuse with the specified fallback, not arbitrary generation or abstention policies. Routing logs identify rejection counts, but precision requires an independent staleness signal, and error comparisons require quality assessment. The daily budgets transport sampled quality parameters across rejection sets; the monthly one-shot analysis instead measures cached and fallback errors directly on the additionally rejected cases. Both report abstention separately because erroneous-delivery counts alone do not measure service utility.

6 Evaluation Design

6.1 Evaluation questions and evidence layers

We evaluate five questions: **RQ1**, agreement between declared availability models and controlled failures; **RQ2**, reuse selectivity, fallback demand, and endpoint quality; **RQ3**, restoration under read loss without bypassing eligibility; **RQ4**, paired routing differences and measured maintenance costs; and **RQ5**, restoration and eligibility-guard behavior on non-design objects.

The evidence comes from three complementary layers.

1. **Controlled audits** use HotpotQA distractor validation (7,405 questions) and MuSiQue development (2,417 answerable questions). Supporting sentences and paragraphs define designated evidence sets for availability and dependence checks.
2. **Daily replay** uses a fast-moving English Wikipedia slice constructed in the spirit of DailyQA [22], two independently constructed generator-specific caches, and source-defined staleness. It measures frozen-cache routing under exact-key requests.
3. **Monthly replay** uses Wikipedia/Wikidata snapshots from 2026-04-01 to 2026-08-01. It executes paraphrase screening, fallback, write-back, and read-loss protection, with human-labeled endpoint quality.

RQ1 concerns the controlled layer; RQ2 spans both replays; and RQ3–RQ5 use the monthly workflow. Agreement with an availability formula checks the model and implementation, not answer correctness.

6.2 Daily frozen-cache replay

Panel and caches.

The pool contains 100 candidate pages: 54 World Cup players, 16 national teams, 14 tournament or season pages, 6 weather-season pages, and 10 stable controls. Twelve pages without in-window edits receive their exact t_0 boundary revision; removing one redirect leaves 99 usable pages. We retain infobox values, lead text, and revision hashes at 21 daily states from 2026-07-01 to 2026-07-21.

Content filtering and topic-proximity matching identify 373 base infobox claims with lead-sentence anchors. Non-overlapping same-page triples yield 99 three-part questions over 297 claims. Each claim records its first qualifying support sentence; the strict contract additionally records one context sentence when available. Repeated sentence identities count once.

Kimi (`kimi-k2-turbo-preview`) and GPT (`gpt-5-mini` via the Poe API) each answer all 99 template-composed questions using t_0 evidence. Set-level lexical acceptance retains 62 Kimi and 68 GPT entries, with 58 accepted by both. Acceptance checks answer-wide occurrence of normalized textual values or extracted numeric substrings. It does not verify their assignment to question parts or semantic support. Downstream metrics condition on the accepted cache of each generator.

Staleness and policies.

A claim is stale when its normalized current infobox value differs from t_0; an answer is source-defined stale when any of its three claims is stale. Sentence survival instead tests normalized substring persistence in the current lead.

The evaluated policies are similarity-only reuse, asserted and strict sentence contracts, KV-claims over the three designated keys, mapped overlap and equality gates, page-hash and infobox-diff references, and TTL-1d/3d/7d/14d. The sentence contracts check stored sentence survival. KV-claims checks the designated values directly. Supplementary Section S9.1 specifies all predicates.

Frozen execution and fallback.

Every policy rechecks the same base entry at each of 20 post-construction ticks, producing 1,240 Kimi and 1,360 GPT answer–tick cases. Each rejection counts as one fallback invocation; no output is written back. TTL-d rejects when age is strictly greater than d days. A previously rejected entry may pass again if its base premises recover; four Kimi and five GPT entries do so on this panel. Frozen-cache routing shares are not maintenance counts under write-back: resetting issuance times or replacing certificates changes both types of trajectories.

Real same-generator fallback is executed only at each entry's first asserted-gate rejection: 19 Kimi and 22 GPT cases. Generation uses the current lead, and lexical acceptance checks available current infobox values. Repeated rejections and comparator rejections contribute routing demand, not a complete archive of generated deliveries.

Metrics and inference.

Routing metrics are interception of source-defined stale cases, fallback demand, rejection among non-stale cases, and stale cached deliveries divided by cache-served cases. The last metric excludes fallback quality and is not an end-to-end error rate.

Policy contrasts use 10,000 paired page-cluster bootstrap resamples with seed 42; each drawn page contributes every accepted answer and evaluation tick of that page, with draw multiplicity. Nominal Wilson intervals are descriptive and do not adjust for dependence among repeated answer–tick cases. Sampled human assessments separately calibrate the projected answer-error budgets; their quality-transport assumptions appear in Supplementary Section S9.2. Service-demand scenarios, which combine routing counts with fixed cost references and exclude state upkeep, appear in Supplementary Section S8. The workload is change-enriched and does not estimate Wikipedia-wide prevalence.

6.3 Monthly replay and comparators

Panel.

The monthly series contains five English Wikipedia snapshots and four Wikidata views, the latter dated 2026-04-01, 2026-05-01, 2026-07-01, and 2026-08-01. Thirty-one property templates yield 6,580 candidate (entity, property) pairs over 1,739 claim-covered entities. A pair is dependency-stale when a designated base value disappears

Table 1 Monthly reuse contracts. Claims and sentences are checked against the named current snapshot.

Contract	Required claims	Required sentences
Strict	All designated	All designated
Majority	All designated	Strictly more than half; true if none designated
Claims only	All designated	None
Asserted claims	Designated values lexically identified in the answer	None

from the target projected eligible-claim view; pure additions do not qualify, and the label describes a dependency change, not whether a cached answer has become wrong. Appendix B.2 specifies the executed claim representation.

We select all 52 stale pairs and 104 intact controls. Lexical acceptance leaves 154 cached entries: 50 stale and 104 intact, over 143 entities. Their records average 2.79 designated claims and 2.46 sentences. The panel is deliberately enriched for dependency staleness; its routing rates are not production-stream estimates.

Screening and contracts.

Each incoming request is a model-generated paraphrase of its cached question. The endpoint replay uses paired-candidate screening with `bge-m3` cosine similarity and threshold $\tau = 0.8364$, accepting on $\geq$. This yields 147/154 hits: 43/50 stale and 104/104 intact. Unified retrieval gives the same matches with no misrouting; entity-grouped five-fold calibration yields 144/154 held-out hits. These within-panel checks do not establish general request compatibility (Appendix B.2).

Table 1 varies dependency scope. Claims use projected claim-membership; sentences use normalized identity. The asserted set is lexical and need not capture every factual assertion. These are distinct contracts, not thresholds of one contract.

Comparators include similarity-only reuse, blind refresh, page/entity/property equality, asserted-value containment in page text, and an LLM recheck judge. The judge evaluates the cached question rather than the incoming paraphrase. Exploratory overlap and equality mappings (Section 2) include a claim-level equality ablation. A crossed gate-by-alias control separates gate choice from alias expansion.

Temporal execution.

The one-shot comparison checks the base cache at 2026-08-01. The multi-tick comparison repeats the 154 requests at 2026-05-01, 2026-07-01, and 2026-08-01, with a separate cache for each policy. Accepted repairs update that policy's response and support records; aliases are added only when enabled. Page-change maintenance rejects when wikitext differs from the cached answer's issuance state.

Every reachable fallback key has an archived model output or an explicit abstention; missing coverage is an execution error. No template substitutes for a

model output in the headline replay. Accepted regenerations count delivered outputs that pass acceptance, not generation attempts or archive-construction calls (Appendix C.1).

The replay regenerates from the cached question. The 151 faithful paraphrases are annotated as intent-equivalent; the three drifted requests abstain in the gated trajectories because usable evidence is absent. Thus fallback under changed query intent is not evaluated.

Maintenance proxies.

For a cached delivery, let A be its lexical asserted-claim set, P its full designated set, and L_t the currently eligible claims before injected read loss. A violating serve has $A \not\subseteq L_t$. A partially-stale serve has $A \subseteq L_t$ but $P \setminus A \not\subseteq L_t$. These disjoint categories measure evidence currency, not answer quality. Zero violations are structural for gates requiring asserted-claim eligibility; blind refresh has none vacuously because it makes no cached delivery.

6.4 Annotation and quality accounting

Two non-author annotators independently assess all 154 cached entries using a bilingual (Chinese–English) sheet containing the cached question, incoming paraphrase, answer, designated claims and extracted sentences with target-period survival marks, and two-period value status of the queried property. They do not consult the source pages or snapshot files, and gate outputs and dependency-staleness labels are withheld. Because the survival marks show which designated claims disappeared, dependency staleness is inferable from the sheet. The assessment is therefore blind to policy decisions but not to the evidence state on which the contracts operate.

Judgments cover base correctness (A1), evidence support (A2), paraphrase fidelity (A3), target-period correctness (A4), and temporal attribution (A5). Pooled agreement is 96.2% over 770 decisions; the first annotator adjudicates disagreements using the operational definitions. Appendix B.3 gives question-specific agreement, raw-label sensitivity, and the regeneration-label protocol.

Delivery categories are correct, incomplete, wrong, indeterminate, and abstain. An answer to the cached question that fails to answer a drifted incoming query counts as wrong for that request. Temporal decay and query drift are separate attribution tags: an entry is decayed when A5 links a target-period wrong or incomplete answer to an invalidated assertion between the two periods, incompleteness from base-period generation gaps or pure value additions is change-independent, and an incoming query is meaning-drifted when A3 marks its paraphrase as unfaithful to the cached question. The panel contains nine decayed entries and three drifted requests, all in the dependency-stale arm. Confirmed-correct hits must answer both the cached and incoming questions correctly at the target state.

Regenerated endpoint answers are labeled using their actual archived text and origin tick, including earlier write-backs. Fault runs can reach additional unlabeled outputs; these remain unassessed, never counted as correct. Confirmed-correct fault-run counts are therefore annotation-based lower bounds, and between-arm differences can reflect label coverage as well as delivery quality.

6.5 Execution, uncertainty, and analytical checks

Faults and paired comparisons.

Reuse-path read faults use stable keys (seed, tick, request, premise, read route). Arms share the same draw for a given key; source and backup routes fail independently. Different requests need not share faults even when they cite the same premise. Each policy evolves its own write-back state, so paired transitions compare trajectories, not protection toggles at an identical cache state.

Entity-cluster bootstrap intervals describe monthly panel comparisons. Means and p95 values across fault realizations describe injected-fault variation; p95 is not a confidence interval for the mean. Appendix C specifies the estimands and resampling procedures. Unless stated otherwise, reported intervals are 95% intervals; bootstrap intervals use empirical percentile endpoints.

Timing and frozen check.

Local timings use one Apple M3 Pro with materialized evidence. Serve-path measurements use 3,000 randomized hit evaluations and exclude query embedding, remote state acquisition, and end-to-end serving. Storage figures count logical payload bytes.

The frozen new-object check retains 50 accepted pairs after testing 181 candidates and uses 100 fault realizations at $p = 0.10$. Fresh-only entries and template fallback make this a restoration and eligibility-guard check, not independent temporal-error or generation-quality validation.

Analysis and protocol map.

Table 2 links the analysis to the evaluation. Appendices A–C provide proofs, data protocols, and execution details. Online Resource 1 gives controlled audits (S1), numerical protection references (S2), monthly extensions (S3–S7), daily service-demand scenarios (S8), daily protocols and quality sensitivity analyses (S9), and the artifact inventory (S10).

7 Experimental Results

We report controlled availability checks, daily frozen-cache routing, and monthly maintenance with endpoint quality assessment. The analyses use different outcome units and are not pooled.

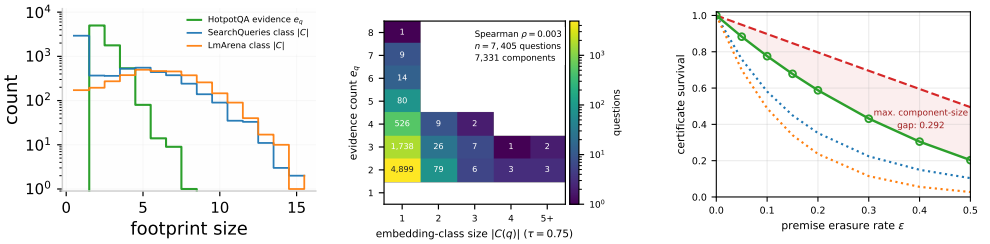
7.1 Contract availability under controlled failures

HotpotQA supporting sentences and MuSiQue supporting paragraphs define conjunctive evidence sets [32, 33]. Unless stated otherwise, simulations use seed 42, 20 repetitions, and erasure rates $\{0, 0.05, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5\}$.

HotpotQA has mean evidence size 2.43. Simulated mean availability differs from $N^{-1}\sum_q (1-\varepsilon)^{e_q}$ by less than 0.001 across the grid. Replacing evidence counts with embedding-component sizes changes predicted availability by up to 0.292 (Figure 3). The components join questions with `all-MiniLM-L6-v2` cosine similarity at least 0.75 [34]. Thus semantic grouping size is not an adequate evidence-footprint proxy on this

Table 2 Role of the analytical results in the evaluation.

Result	Evaluation use	Boundary
Definition 3.1	Snapshot checks and restoration guards	Permission and index predicates held fixed
Lemma 3.1; Proposition 5.1	Contract checking and separate verification audits	Relative to supplied records and named state
Lemma 4.1; Theorem 4.1	Availability audits, small-DAG enumeration, static protection models	Workload audits use flat sets; no general DAG serving benchmark
Corollary 4.2	Fixed-state reference for read protection	Not separately audited per check
Corollary 4.1; N^*	Storage references	No executed placement uses N^*
Proposition A.2	Majority contract and heuristic families	Families not evaluated under read faults
Theorem A.1; Remark A.1	Numerical coding references	No encoder–decoder execution
Proposition A.1	Footprint-substitution and count-perturbation audits	Bounds count-model differences only
Proposition 5.2; Corollary 5.1; Remark 5.1	Daily error projections and monthly one-shot accounting	Daily quality is transported; monthly shared-output identity is evaluated directly

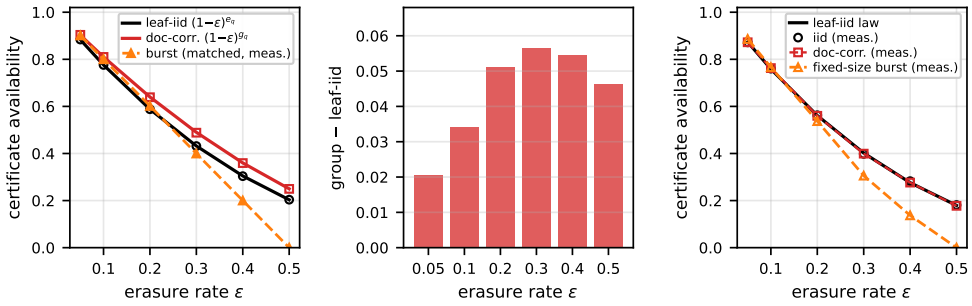


(a) Footprint distributions. (b) Joint footprint distribution. (c) Annotation-based erasure audit.

Fig. 3 Declared evidence footprints and synthetic class-size references. (a) HotpotQA evidence counts and vCache benchmark class sizes. (b) Evidence size versus embedding-component size at $\tau_c = 0.75$. (c) Independent-erasure availability and means of 20 simulations. Trace curves use full class sizes as synthetic exponents, not measured evidence demand.

workload. The figure also includes synthetic class-size references from the first 20,000 rows of each released vCache benchmark [2, 6]; these exponents are not measured premise requirements.

Dependence changes availability even when erasure marginals match. Independent failures of g_q supporting groups give $(1-\varepsilon)^{g_q}$. The burst arm removes $a_q = \max\{1, \lfloor g_q/2 \rfloor\}$ groups with probability $\min\{\varepsilon/(a_q/g_q), 1\}$. HotpotQA averages 2.00 supporting documents and 2.43 sentence leaves. At $\varepsilon = 0.3$, independent-document availability exceeds independent-sentence availability by approximately 0.058. At $\varepsilon = 0.5$, the matched-marginal burst arm has zero availability, versus 0.203 and 0.25 under independent sentence and document failures (Figure 4). The matched-marginal results illustrate why dependent availability depends on $\Pr[D_{\exp}(q, S) \subseteq \widetilde{B}]$, not on erasure marginals alone.



(a) HotpotQA: correlated erasure. (b) HotpotQA: group–leaf availability gap. (c) MuSiQue control.

Fig. 4 Query-local leaf, group, and burst failures (20 repetitions, seed 42). (a) HotpotQA availability. (b) Independent-group minus independent-leaf availability. (c) MuSiQue, where group and leaf counts coincide. Burst marginals match the nominal rate only before capping; MuSiQue achieves marginal 0.448 at nominal $\varepsilon = 0.5$.

For MuSiQue, group and leaf counts coincide. Its capped burst mechanism achieves mean erasure marginal 0.448 at nominal $\varepsilon = 0.5$, so that point is not a matched-rate comparison. All draws are query-local. The audits therefore measure mean per-query availability, not joint reliability under a shared corpus failure.

Closure and exposed-leaf checking agree in all 990,560 states enumerated exhaustively over 400 generated conjunctive DAGs, an implementation check of Lemma 4.1.

An instrumented retrieval audit over 66,581 title-deduplicated HotpotQA records agrees with the declared model's predictions within 0.0016; at depth 10, 90.2% of questions retrieve at least one gold title and 38.7% all of them. Supplementary Sections S1 and S2 give the protocols and the numerical coding and shared-root references.

7.2 Daily replay: routing and projected answer error

Routing and rejection selectivity.

The accepted Kimi and GPT caches contain 62 and 68 entries, yielding 1,240 and 1,360 answer–tick cases. Each contains 53 source-defined stale cases: prevalence is 4.27% and 3.90%, respectively.

Table 3 reports frozen-cache routing. The asserted sentence contract intercepts $52/53 = 98.11\%$ of stale cases in each cache (nominal Wilson interval $[90.1, 99.7]\%$). Fallback demand is 18.6% for Kimi and 19.7% for GPT, with served-conditional stale rates of 0.10% and 0.09%. Policy contrasts use paired page-cluster bootstrap (10,000 resamples, seed 42; resampled pages contribute with their draw multiplicity). TTL-14d instead rejects 30.0% of cases and has stale rates of 3.46% and 3.15%. Paired page-cluster intervals for the asserted-minus-TTL stale-rate difference are $[-7.36, -0.46]$ and $[-6.81, -0.41]$ percentage points. Against TTL-1d, which rejects 95% of cases and serves no stale case, the fallback-demand differences are $[-86.4, -65.2]$ and $[-84.3, -64.8]$ points.

Table 3 Daily frozen-cache routing. Kimi: 62 entries, 1,240 answer–tick cases; GPT: 68 entries, 1,360 cases; each contains 53 source-defined stale cases. Stale percentage conditions on cache-served cases; fallback percentage uses all cases. Bold marks the lowest fallback demand among policies with zero stale cached delivery in each cache. KV-claims achieves this result by monitoring the values that define benchmark staleness. Predicates and intervals appear in Supplementary Section S9.1; service-demand scenarios in Section S8.

	Kimi cache		GPT cache	
arm	stale %	fallback %	stale %	fallback %
similarity-only	4.27	0.0	3.90	0.0
TTL-1d	0.00	95.0	0.00	95.0
TTL-3d	0.54	85.0	0.49	85.0
TTL-7d	2.07	65.0	1.89	65.0
TTL-14d	3.46	30.0	3.15	30.0
page-hash reference	0.00	81.2	0.00	81.8
GC-overlap mapping	0.00	81.2	0.00	81.8
FC-equality mapping	0.00	81.2	0.00	81.8
infobox-diff (ref.)	0.00	58.0	0.00	59.6
PVCacheServe sent-asserted	0.10	18.6	0.09	19.7
PVCacheServe sent-strict	0.10	21.6	0.10	22.6
PVCacheServe KV-claims	0.00	**4.3**	0.00	**3.9**

Table 4 Daily routing ledger (Kimi/GPT). Fallbacks per intercepted stale case equal rejection counts divided by intercepted source-defined stale cases. FR|valid is rejection among non-stale cases. Excess demand is fallback share minus stale prevalence (4.27%/3.90%). KV-claims attains the zero-stale-delivery floor by construction (Remark 7.1). Counts derive from Table 3.

arm	fallback/intercepted	FR|valid (%)	excess demand (pp)
KV-claims (oracle-aligned)	1.0/1.0	0.0/0.0	0.0/0.0
sent-asserted	4.4/5.2	15.1/16.5	14.3/15.8
sent-strict	5.2/5.9	18.2/19.6	17.3/18.7
infobox-diff (ref.)	13.6/15.3	56.1/57.9	53.7/55.7
page-hash = GC = FC	19.0/21.0	80.4/81.1	76.9/77.9
TTL-1d	22.2/24.4	94.8/94.8	90.7/91.1
TTL-14d	16.2/17.7	29.4/29.5	25.7/26.1

Each sentence-gated cache misses one stale case because its support sentence persists after the infobox value changes. Conversely, sentence changes while values remain intact produce non-stale rejection rates of 15.1% and 16.5%. These are disagreements between textual persistence and the benchmark's structured staleness definition.

Table 4 exposes the cost of these disagreements. For Kimi, the asserted gate rejects 179 non-stale cases while intercepting 52 stale cases. It therefore requires 4.4 fallback invocations per intercepted stale case, compared with 5.2 for GPT and 22.2/24.4 for TTL-1d.

The aggregate routing points are similar across generators. On the 41 shared pages, the Kimi-minus-GPT stale-rate interval is $[-0.01, +0.03]$ percentage points. This does not establish equivalence. Gate verdicts are deterministic conditional on identical support and state, whereas generation affects cache acceptance, composition, and fallback quality.

Dependency scope.

Page-hash, GC-overlap, and FC-equality have identical observed routing: 81.2% and 81.8% fallback demand. On this panel, the mapped additional predicates pass whenever the page-version condition passes. Against GC-overlap, the asserted gate's fallback-demand difference has page-cluster intervals of $[-74.3, -49.6]$ percentage points for Kimi and $[-72.8, -50.2]$ for GPT.

KV-claims monitors only the three designated infobox keys. It rejects exactly the 53 stale cases, with no stale cached delivery and 4.3%/3.9% fallback demand. Infobox-diff monitors every key and rejects 58.0%/59.6%. The difference isolates monitored scope within the same structured source representation.

Remark 7.1 (A fallback-demand floor for zero stale cached delivery). In the daily reuse-or-fallback action space, zero stale cached delivery requires rejecting every source-defined stale case. Fallback demand is therefore at least stale prevalence. KV-claims attains this floor because its monitored values define benchmark staleness; this is alignment by construction, not independent detection.

Under Proposition 5.2,

$$E_{\mathrm{pol}} - E_{\mathrm{KV}} = (\rho - \rho_s)(ar - b) + m(1 - ar).$$

When $ar > b$, rejecting non-stale cases and missing stale cases both weakly increase expected error relative to rejecting exactly the stale cases.

The structured contract avoids sentence-rewriting declines when current key values are trustworthy and key equality is the required contract. Sentence persistence provides a different, imperfect currency signal on this infobox-anchored workload.

Acceptance quality and projected error.

Routing selectivity does not establish better delivered answers. At first asserted-gate rejection, real fallback generation produces 19 Kimi and 22 GPT outputs; lexical acceptance retains 17 and 20. In a double-annotated sample of 20 outputs, only 9 are adjudicated correct (45.0%, nominal Wilson interval $[25.8, 65.8]$%). Annotators agree on 19/20 cases ($\kappa = 0.90$). Of 19 lexically accepted outputs in the sample, 10 are wrong or incomplete. Seven failures involve values mentioned in the wrong answer part, which set-level matching does not detect.

A stratified audit of 40 accepted base answers finds 18 wrong or incomplete. Weighting the flagged and clean strata gives estimated cached error $b = 22.8\%$. The fallback sample gives erroneous-delivery mass $ar = 10/20 = 0.50$. Table 5 applies these estimates to observed routing under Proposition 5.2. It assumes every source-defined stale cached delivery is erroneous and transports quality across policies and ticks.

Under this model, Kimi similarity-only reuse has 261.0 erroneous deliveries per 1,000 cases, compared with 239.6 for KV-claims, 279.3 for the asserted gate, and 486.4 for TTL-1d. Thus fewer stale cached deliveries need not mean fewer errors: replacement quality can outweigh interception. The asserted gate's rejection precision is 0.225, versus 0.194 for GPT; its point-estimate fallback quality lies outside the improvement window in both caches.

Table 5 Projected erroneous deliveries per 1,000 Kimi answer–tick cases under Proposition 5.2. Inputs are $b = 22.8\%$, $a = 19/20$, and $r = 10/19$. Wrong and incomplete deliveries count as errors; abstentions do not. These are homogeneous-quality projections, not fully annotated delivery totals. GPT budgets and transport assumptions appear in Supplementary Section S9.2.

policy	π	stale	cached error	fallback error	total	abstain
similarity-only	–	42.7	218.3	0.0	261.0	0.0
KV-claims	1.000	0.0	218.3	21.4	239.6	2.1
sent-asserted	0.225	0.8	185.3	93.1	279.3	9.3
page-hash	0.053	0.0	42.8	406.0	448.8	40.6
TTL-1d	0.045	0.0	11.4	475.0	486.4	47.5

Per-part re-scoring, without new generation, rejects seven of the ten erroneous accepted fallbacks and one correct output. The sampled erroneous-delivery mass falls to $3/20 = 0.15$. It also flags all 18 non-correct base answers and two correct answers in the base audit. At the clean-stratum point estimate $b = 0$, the transported asserted-gate budget improves on similarity-only reuse. The clean sample contains 0/20 errors, with Wilson upper bound 16.1%. At that upper-end setting, $ar < b$, and more aggressive rejection has lower projected error. Joint uncertainty in cached and fallback quality also leaves the comparison with similarity-only reuse unresolved. Supplementary Section S9.2 reports the sensitivity budgets and transport assumptions. A separate routing replay retains only per-part-accepted base entries: 46 Kimi and 51 GPT. Asserted-gate fallback demand becomes 16.7%/17.1%, with precision 23.4%/25.9%. Served-conditional stale rates are 0.13%/0.12%. These results describe restricted caches, not the service cost or quality of the excluded requests.

Fallback demand versus elapsed cost.

Fallback demand is the robust cost measure: the asserted gate issues 186/197 fallback invocations per 1,000 cases, versus 950 for TTL-1d. Elapsed-time comparisons depend on how current state is obtained. With maintained audit state, nominal demand is 208/753 s per 1,000 cases versus 1,062/3,629 s for TTL-1d. If every state-dependent check instead re-reads the source page (1.775 s), TTL-1d is cheaper for Kimi. Supplementary Section S8 gives the cost model and both conventions; state upkeep is not measured.

7.3 Monthly replay: maintenance, quality, and protection

Reuse decisions.

Among 147 similarity hits, eight are temporally decayed, three answer meaning-drifted requests, 118 are confirmed correct, twelve are change-independent incomplete, and six are indeterminate. All four contracts decline the eight decayed and three drifted hits (Table 6). The drifted requests fail because required evidence is unavailable, not because the gate detects intent change.

Confirmed-correct declines fall from 38/118 under strict reuse to 25/118 under claims-only and 18/118 under asserted claims. Entity- and property-equality checks decline more correct hits; the LLM judge declines only 6/118 but serves three decayed answers. These are panel-specific scope–coverage trade-offs.

Table 6 Monthly reuse decisions on 147 similarity hits. Correct-hit declines use 118 confirmed-correct hits. Bold marks the fewest confirmed-correct declines among mechanisms serving no decayed or drifted hit. All four contracts decline the eight decayed and three drifted hits; drifted hits fail for evidence reasons, not detected intent change. An exact validator with the same asserted dependencies is decision-equivalent to the asserted-claims gate.

Mechanism	Served	Decayed served	Drifted served	Correct hits declined
Similarity-only	147	8	3	0/118
Strict contract	89	0	0	38/118
Claims + majority sentences	96	0	0	33/118
Claims only	104	0	0	25/118
Asserted claims only	111	0	0	**18/118**
Page unchanged	43	1	0	82/118
Entity view unchanged	56	0	0	64/118
Property claims unchanged	100	0	0	25/118
Asserted values in target page	97	3	3	35/118
LLM recheck judge	129	3	0	6/118

One-shot delivery.

Unlike the daily projections, the one-shot comparison directly labels shared archived fallback outputs. Similarity-only reuse delivers 23 wrong or incomplete answers. Strict reuse rejects 58 hits containing 15 cached errors and introduces eight fallback errors, giving $23 - 15 + 8 = 16$. Asserted claims rejects 36 hits containing 14 cached errors and introduces seven fallback errors, also giving 16. Every contract has a lower fallback error rate than cached error rate on its additionally rejected set (Remark 5.1; Supplementary Table S4.5).

Strict one-shot delivery contains 123 correct answers and no annotated-wrong answer, versus 119 correct and six wrong under similarity-only reuse. Abstentions increase from six to fourteen. The correct-delivery difference is 2.6 percentage points, with entity-cluster interval $[-3.9, +9.3]$, so the comparison does not establish a correct-delivery gain.

Maintenance and endpoint quality.

Across 462 request-ticks, the asserted loop uses 29 accepted regenerations, versus 421 for blind refresh, 53 for strict reuse, and 174 for page-change maintenance. The mapped page-version overlap and equality gates use 181 and 187. Claim-level equality uses 43 with aliases disabled and 41 with aliases enabled, versus 31 and 29 for asserted claims.

The alias control isolates scope: property equality also reacts to additions and changes in unasserted values, whereas alias expansion changes accepted regenerations by two for asserted claims and claim equality and by zero for the page-version mappings (Supplementary Section S4.5). All zero-violation configurations abstain on 40–41 request-ticks.

Table 7 Human-labeled monthly endpoint quality over 154 requests. Primary-generator categories are mutually exclusive and sum to 154. Earlier write-backs are assessed using their actual archived answers at the endpoint. The final columns report second-generator trajectory outcomes; base-cached answers remain unchanged. Bold marks the highest observed correct-delivery count for each generator, not statistical superiority.

	Primary generator					Second generator	
Policy	Correct	Incompl.	Wrong	Indet.	Abstain	Correct	Wrong
Static cache	119	17	6	6	6	119	6
Blind refresh	119	21	0	0	14	105	1
Page-change loop	121	19	0	1	13	109	2
Strict loop	122	17	0	1	14	115	0
Asserted loop	**124**	14	0	2	14	**120**	0

Table 7 labels actual endpoint answers, including earlier write-backs. The primary asserted loop delivers 124 correct answers versus 119 for blind refresh, with no annotated-wrong delivery under either policy. The refresh-minus-asserted difference is -5 answers (entity-cluster interval $[-13.2, +3.0]$); superiority and non-inferiority are not established.

Second-generator re-execution retains the base cache and evidence states but rebuilds certificates and trajectories from the new outputs. Accepted-regeneration totals are unchanged for all five policies. Correct deliveries decrease in every maintenance arm; the refresh-minus-asserted difference becomes -15 ($[-24.2, -6.2]$). Neither gated loop has an annotated-wrong multi-tick endpoint delivery under either generator. This does not extend to second-generator one-shot strict reuse, which has one wrong regenerated delivery. Supplementary Section S4.4 reports complete categories and labeling.

Read-loss protection.

Table 8 reports 200 coupled fault realizations per cell. At $p = 0.10$, fixed protection reduces mean accepted regenerations from 68.6 to 34.2 for asserted claims and from 158.0 to 75.2 for strict reuse. Its counts are below the random-placement comparator in every reported cell.

Fixed placement covers all designated payloads of 115 base entries. For eligible, fully covered certificates, Corollary 4.2 leaves only double-route read loss; write-back keys outside the base universe remain unprotected. A separate guard path reads protected ineligible objects and rejects matching payloads before injected read faults are evaluated. These probes are distinct from loss-triggered restoration. Trusting base copies without eligibility checking instead serves all eight decayed hits.

The asserted contract's paired reduction is $(7{,}823 - 942)/200 = 34.405$ accepted regenerations per run. Reverse transitions occur because write-back states differ across arms; they do not contradict fixed-state monotonicity. Local backup reads average $4.9\,\mu$s, excluding fingerprint and eligibility checks. Fault-run outputs without quality labels remain unassessed (Supplementary Section S7).

Table 8 Accepted regenerations over three monthly update ticks: mean and p95 (parentheses) over 200 coupled fault realizations. Bold marks the lowest mean among none, random, and fixed protection for each contract and read-loss rate. Fixed placement uses 58,954 logical payload bytes. The random comparator shuffles the payload-object universe and greedily fills the same byte budget, retaining its placement across ticks. Source and backup reads fail independently. Full is an ideal read-any reference and is excluded from the boldface comparison. Erasure-free totals are 29 for asserted claims and 53 for strict reuse.

contract	protection	$p = 0.05$	$p = 0.10$	$p = 0.20$
asserted	none	51.2 (59)	68.6 (78)	97.7 (107)
	random	32.1 (35)	37.4 (42)	52.2 (60)
	fixed	**30.3** (33)	**34.2** (38)	**47.4** (54)
	full (ref.)	29.0 (29)	29.0 (29)	29.0 (29)
strict	none	115.7 (125)	158.0 (169)	217.0 (228)
	random	64.2 (71)	81.2 (90)	123.6 (138)
	fixed	**60.8** (65)	**75.2** (82)	**115.2** (126)
	full (ref.)	53.0 (53)	53.0 (53)	53.0 (53)

Non-design-object mechanism check.

On 50 accepted non-design pairs and 100 fault realizations at $p = 0.10$, fixed protection reduces mean accepted regenerations from 16.8 to 1.9. The paired reduction is 14.9, with fault-resampling interval $[14.1, 15.7]$. Fresh-only entries and template fallback make this a restoration mechanism check; Section 8.3 states its scope.

The monthly replay demonstrates the executed maintenance lifecycle under labeled outputs. Its accepted-regeneration counts are not directly comparable to daily frozen-cache fallback demand, and its endpoint-quality ordering remains generator-dependent.

8 Discussion and Limitations

8.1 What the framework establishes

PVCacheServe makes reuse validity explicit relative to accepted support and a named audit state. Exact checking establishes certificate soundness and family-completeness within that representation. The residual-premise law then characterizes fixed-state availability under independent source erasures. Neither result establishes semantic sufficiency, request compatibility, or state-provider accuracy.

The experiments separate these boundaries. Controlled audits check availability calculations and failure semantics. Daily replay measures disagreements between sentence persistence and structured staleness. Monthly replay evaluates maintenance with actual archived outputs. Table 9 summarizes the observed failure patterns without treating them as an exhaustive causal taxonomy.

Nested required sets imply nested acceptance sets only at a common state. They imply neither better answer quality nor fewer multi-tick regenerations: fallback quality varies, and write-back changes later states. Similarly, the selective-regeneration identities do not require an exact certificate gate. They explain error differences from routing and cached/fallback losses. Exact checking supplies an explicit reuse criterion, not an independent guarantee of end-to-end improvement.

Table 9 Observed failure patterns and their interpretation. The categories describe retained cases rather than exhaustive causal attribution.

Observation	Interpretation
Support text persists after a value change	Contract–truth mismatch: one missed stale case per daily cache
Sentence requirements decline correct answers	Dependency scope changes coverage: monthly declines 38 → 25 → 18 from strict to claims-only to asserted claims
Three paraphrases ask for a different relation	Matching failure; gate declines arise from absent evidence, not intent-drift detection
Set-level matching admits wrong-part values	Construction and fallback acceptance require more than answer-wide lexical inclusion
Endpoint completeness changes with the generator	Maintenance counts do not determine answer-quality ordering
Restored bytes match but are no longer eligible	Payload integrity cannot substitute for current eligibility

8.2 Implications for deployment

The results suggest four practices. First, record at issuance the dependencies an answer actually asserts: on the monthly panel this scope declines every decayed hit while declining the fewest correct ones. Second, choose dependency scope by its fallback cost rather than its interception rate: KV-claims attains the daily fallback floor because its monitored keys define staleness, whereas sentence persistence also rejects rewritten but valid evidence. Third, calibrate fallback before widening rejection. Where shared fallback outputs exist, Remark 5.1 compares cached and fallback losses directly on the additionally rejected set; otherwise the operating window (Proposition 5.2) is a sensitivity model whose fallback parameter must be estimated over all invocations, including abstentions, because accepted outputs alone estimate r rather than ar. Under that model, $ar < b$ favors broader rejection and $ar > b$ makes precision decisive. Fourth, treat backups as read repair only: restored bytes re-enter checking as candidates, never as eligibility.

All state-dependent gates require current evidence. Maintained state avoids per-request source reads, but its upkeep is not measured here, and the local timings describe operation costs under materialized state. The experiments exercise superseded claims and changed sentences, not permission revocation; the envelope audit checks field binding under an honest issuer, not adversarial authentication.

8.3 Threats to validity

Workload and matching.

The daily pool is change-enriched, infobox-anchored, and limited to 21 snapshots. It repeats exact-key requests against a frozen cache; its fallback shares are not write-back maintenance counts. The monthly panel is staleness-enriched and contains templated single-property questions over 143 entities. Paraphrase screening and aliasing are evaluated there, but general cross-query support transfer is not.

Quality assessment.

Daily error projections use 40 sampled base answers and 20 sampled first-rejection fallbacks. They transport quality across policies and ticks and treat source-defined stale cached delivery as erroneous. Per-part sensitivity budgets additionally transport the original routing inputs; the restricted-cache replay measures routing separately but excludes removed requests. Monthly labels use evidence extracts from which dependency staleness is inferable. The second-generator comparison retains the primary base cache and changes both the requested model and potentially its backend reasoning configuration. Intervals condition on archived outputs, not repeated-generation variability. Fault-run outputs without labels remain unassessed.

Execution and scope.

The prototype uses local stores, materialized snapshot state, and injected reuse-path read faults. Fallback evidence acquisition is not subjected to the same faults. Remote state acquisition, complete upkeep costs, authenticated issuance, and concurrent updates are outside the evaluation. The frozen new-object check uses fresh entries and template fallback, so it validates restoration mechanics rather than temporal answer quality. Workload experiments do not evaluate general DAG serving, arbitrary family search, shared-root execution, coded decoding, or optimal protection placement.

9 Conclusion

PVCacheServe separates three questions that similarity-only caches conflate: whether a cached response matches a request, whether its declared evidence is still available and eligible, and whether it is correct. The residual-premise law prices availability at a fixed state; the selective-regeneration identities price the decision to replace a cached answer.

Three empirical lessons follow. Dependency scope sets maintenance cost: the asserted-claims contract declines every decayed monthly hit while needing 29 accepted regenerations against 421 for blind refresh. Interception does not set quality: on the daily replay, fallback acceptance rather than staleness dominates the projected error budget. Eligibility-checked protection reduces read-loss regenerations without reinstating superseded evidence.

Broader claims require support construction beyond lexical matching, calibrated fallback acceptance, maintained current state with measured upkeep, and evaluation under online updates.

Supplementary information. Supplementary material (Online Resource 1) accompanies this article and is publicly archived in the repository identified below; it contains extended audits, case analyses, exploratory support rules, and the complete experiment inventory.

Reproducibility. The artifact separates replay from archived inputs, re-analysis of retained outputs, and reconstruction from historical sources. Archived model outputs support monthly maintenance replay without new model calls, subject to coverage

checks. The repository includes rebuilt snapshot inputs and a reconciliation report for the descriptive audit. Full Wikimedia dumps are not bundled. Appendix D and Online Resource 1 identify the inputs, execution scripts, and authoritative result fields.

Acknowledgements. The author thanks Hu Xu for assistance with the experiments, and Ze Zhao and Tonghe Shi for their careful annotation and adjudication work.

Statements and Declarations

- **Funding.** No funding was received for conducting this study.
- **Competing interests.** The author declares no competing interests relevant to the content of this article.
- **Ethics approval and consent to participate.** The study uses only publicly released datasets and Wikimedia dumps. Two annotators, neither of whom is the author, participated voluntarily and provided informed consent. They assessed publicly sourced evidence extracts, and the annotation task collected no sensitive personal information about the annotators. The task involved no intervention and collected no personal data beyond annotation judgments, so it did not require formal ethics review.
- **Consent for publication.** Not applicable.
- **Data availability.** Source datasets and snapshot protocols are specified in Appendix B; retained result artifacts and annotation files are publicly available at https://github.com/xuhu6736/proof-valid-benchmarking; all artifacts are indexed in Appendix D and the supplementary material.
- **Code availability.** The experiment scripts, together with the archived model outputs that enable replay without new model calls, are deposited in the same repository.
- **Use of generative AI.** Large language models were used as experimental components as described in Sections 6.2–7.3; generative AI tools were also used for language editing, and the author reviewed and takes full responsibility for the content.
- **Author contributions.** J.X. conceived the study, developed the theory, designed the experiments, analyzed the results, and wrote the manuscript.

Appendix A Analytical Details and Scope

A.1 Proxy sensitivity

Proposition A.1 (Proxy-sensitivity bound)**.** *Let $e, \widehat{e} \in \mathbb{Z}_{\geq 0}$ be exposed-premise counts at the same $0 \leq \varepsilon < 1$. Write $R = (1-\varepsilon)^{e}$, $\widehat{R} = (1-\varepsilon)^{\widehat{e}}$, and $\Delta = |\widehat{e} - e|$. Then*

$$\begin{aligned} \frac{\widehat{R}}{R} &= (1-\varepsilon)^{\widehat{e}-e}, \\ |\widehat{R} - R| &= (1-\varepsilon)^{\min\{e,\widehat{e}\}} \left[1 - (1-\varepsilon)^{\Delta}\right] \\ &\leq 1 - (1-\varepsilon)^{\Delta} \leq \min\{1, \varepsilon\Delta\}. \end{aligned} \tag{A1}$$

For $0 < \delta < 1$, the reliability decisions agree whenever

$$|R - (1-\delta)| > 1 - (1-\varepsilon)^{\Delta}.$$

Proof Factor out $(1-\varepsilon)^{\min\{e,\widehat{e}\}}$ and apply Bernoulli's inequality. The strict-margin condition prevents the bounded perturbation from crossing the reliability threshold. □

For weights $w_q \geq 0$ with $\sum_q w_q = 1$,

$$|\widehat{\overline{R}} - \overline{R}| \leq \sum_q w_q \left[1 - (1-\varepsilon)^{\Delta_q}\right] \leq \min\left\{1, \varepsilon \sum_q w_q \Delta_q\right\}.$$

The bound compares declared count models; it does not validate their semantic support.

A.2 Alternative conjunctive certificates

Definition A.1 (Support function and certificate family)**.** Fix a request–response pair and a finite nonempty family of accepted nonempty premise sets $\mathcal{P}_q = \{P_j : j \in J_q\}$. Each set defines an AND skeleton $G_{q,j}$, with premise leaves exactly P_j, all reaching the service sink q.

Let A_b indicate source survival of premise b. For fixed eligible protection S, define $f_q(A; S) = 1$ when at least one skeleton certifies the response. Skeletons are checked separately; their dependency rules are not combined. Shared premise identities use the same survival indicator.

For each skeleton, let

$$D_j(S) = D_{\exp}(q, S; G_{q,j}), \qquad e_j(S) = |D_j(S)|. \tag{A2}$$

Lemma 4.1 gives

$$f_q(A; S) = \bigvee_{j \in J_q} \bigwedge_{b \in D_j(S)} A_b,$$

with an empty conjunction equal to true. Thus the inclusion-minimal distinct sets $D_j(S)$ are the minimal source-survival requirements after protection, not necessarily the original premise sets. Family reliability is

$$R_q(\varepsilon; S) = \Pr[\exists j \in J_q : D_j(S) \subseteq \{b : A_b = 1\}]\,. \tag{A3}$$

Proposition A.2 (Certificate-family reliability bounds)**.** *Let* $e_{\min} = \min_j e_j(S)$. *Under independent source erasures with* $0 \le \varepsilon < 1$,

$$\begin{aligned}(1-\varepsilon)^{e_{\min}} &\le R_q(\varepsilon; S) \\ &\le \min\left\{1, \sum_j (1-\varepsilon)^{e_j(S)}\right\} \\ &\le \min\left\{1, |J_q|(1-\varepsilon)^{e_{\min}}\right\}.\end{aligned} \tag{A4}$$

A single skeleton attains the lower bound.

Proof The family event contains the survival event of a minimum-size residual set. The upper bounds follow from the union bound and $e_j(S) \ge e_{\min}$. □

The companion extends the residual-leaf law to non-unique AND–OR derivations [5]. Proposition A.2 records only the bounds used here, for families whose branches are declared rather than derived.

A.3 The fixed-split transparent–coded hybrid

Packets have equal cost c_B, and the packet alphabets support the required systematic MDS codes. For $E_n \sim \text{Binomial}(n, \varepsilon)$, define

$$r^*_{\text{MDS}}(n, \varepsilon, \delta) = \min\{r \in \{0, \dots, n\} : \Pr[E_n \le r] \ge 1 - \delta\}.$$

Set $r^*_{\text{MDS}}(0, \varepsilon, \delta) = r^*_{\text{MDS}}(n, 0, \delta) = 0$.

Theorem A.1 (Fixed-split MDS hybrid protection)**.** *Fix* $\kappa \ge 1$ *designated payload packets. Store* $x \in \{0, \dots, \kappa\}$ *transparently and* $r \in \{0, \dots, \kappa - x\}$ *MDS parity packets for the remainder. Assume reliable protected storage, independent source erasures, and no additional payload-dependent decoder information. Recovery means unique reconstruction for arbitrary payload contents.*

Then

$$P_{\text{hyb}}(x, r, \varepsilon) = \Pr[E_{\kappa - x} \le r], \tag{A5}$$

and the minimum storage for reliability $1 - \delta$ *at fixed* x *is*

$$\sigma^*_{\text{hyb}}(x, \varepsilon, \delta) = [x + r^*_{\text{MDS}}(\kappa - x, \varepsilon, \delta)]\, c_B, \tag{A6}$$

where $0 < \delta < 1$ *and* $0 \le \varepsilon < 1$.

If $x_\kappa/\kappa \to \eta \in [0,1]$, *with fixed* $0 < \varepsilon, \delta < 1$, *then*

$$\frac{\sigma^*_{\mathrm{hyb}}(x_\kappa, \varepsilon, \delta)}{\kappa c_B} \longrightarrow \eta + (1-\eta)\varepsilon. \tag{A7}$$

Proof Put $n = \kappa - x$. After e residual erasures, $n - e + r$ code symbols remain. The MDS property gives unique recovery exactly when $e \le r$. For $e > r$, fewer than n symbols cannot determine arbitrary payload contents. The case $n = 0$ is immediate. This proves Eqs. (A5) and (A6).

For fixed ε, δ, Chebyshev's inequality places the $(1-\delta)$-quantile of E_n within $O(\sqrt{n}+1)$ of $n\varepsilon$. Consequently,

$$r^*_{\mathrm{MDS}}(n, \varepsilon, \delta) = n\varepsilon + O(\sqrt{n}+1).$$

With $n_\kappa = \kappa - x_\kappa$,

$$\frac{\sigma^*_{\mathrm{hyb}}}{\kappa c_B} = \varepsilon + (1-\varepsilon)\frac{x_\kappa}{\kappa} + O\left(\frac{\sqrt{n_\kappa}+1}{\kappa}\right),$$

which proves Eq. (A7). □

Remark A.1 (Pure-parity dominance within the MDS hybrid family). Fix total budget $b_{\mathrm{tot}} = x + r \le \kappa$. Under a common erasure realization, $E_\kappa = E_x + E_{\kappa - x}$ and $E_x \le x$, so

$$\{E_{\kappa-x} \le b_{\mathrm{tot}} - x\} \subseteq \{E_\kappa \le b_{\mathrm{tot}}\}.$$

Provided both codes exist, pure parity is at least as reliable as the split. Hence

$$r^*_{\mathrm{MDS}}(\kappa, \varepsilon, \delta) \le x + r^*_{\mathrm{MDS}}(\kappa - x, \varepsilon, \delta).$$

Thus $x = 0$ is an optimal split within this equal-cost family.

A.4 Payload-recovery scope

Parity is decoder state, not a support assertion. Recovered payloads enter certificate checking only after the remaining eligibility conditions pass. Variable-length evidence requires packetization and accounting for padding and metadata. Any payload-dependent decoder information, including hashes, must also be charged.

The companion compares transparent storage with a separate coded payload-recovery benchmark [5]. Theorem A.1 specializes this comparison to a fixed transparent fraction of a designated payload; it does not cover loss-adaptive selection among alternative certificates. No coded decoder or shared-root system is executed here.

Appendix B Data and Annotation Protocol

B.1 Snapshot extraction and projected claim identity

English Wikipedia snapshots are dated 2026-04-01, 2026-05-01, 2026-06-01, 2026-07-01, and 2026-08-01. Wikidata views use the same dates except June [35]. A uniform sample of 2,000 TemporalWiki subject titles (seed 42) yields 1,883 pages recovered at

every Wikipedia snapshot, 1,749 with usable sentences, and 1,739 mapped Wikidata items with claims at every selected state [18]. Rates condition on their stated recovery subsets.

Sentence extraction removes markup and retains normalized sentences of at least 40 characters. A base-list entry is lost when its normalized string is absent from the target sentence set. Repeated base entries retain their multiplicity.

The following projection describes the descriptive snapshot audit. The executed monthly replay uses the implementation-specific representation specified in Appendix B.2. For each property, the truthy projection selects preferred-rank statements when present and normal-rank statements otherwise; deprecated statements are excluded. Only value snaks with non-null `datavalue` are compared. Identity consists of the property and the implemented value projection: `id`, `text`, `time`, or `amount`, in that order of availability. Rank controls selection but is not part of identity. Qualifiers, references, and unretained value attributes are omitted. Claim loss therefore means disappearance of a projected property–value pair, not full-statement deletion. The descriptive claim-loss measure excludes `P421`.

WikiFactDiff reference.

The supplementary descriptive analysis also uses the legacy WikiFactDiff split covering changes between 2021-01-04 and 2023-02-27 [19]. An instance is counted as a value replacement when `is_replace` is true and both obsolete and new object labels are present. These records provide a separate historical change reference and do not enter the monthly serving replay.

B.2 Panel construction and similarity calibration

The 31 property templates are `P6`, `P17`, `P19`, `P20`, `P27`, `P35`, `P39`, `P50`, `P54`, `P57`, `P58`, `P69`, `P86`, `P102`, `P106`, `P123`, `P131`, `P136`, `P159`, `P166`, `P175`, `P264`, `P361`, `P495`, `P527`, `P570`, `P571`, `P577`, `P641`, `P1082`, and `P2046`. They yield 6,580 candidate pairs. All 52 dependency-stale pairs and 104 intact controls are selected; lexical acceptance retains 50 and 104, respectively.

Value labels are resolved from the corresponding snapshots: 248/251 at base and 247/250 at target. Unresolved values retain their `Q`-numbers. Base answers and incoming paraphrases use `kimi-k2-turbo-preview`, with reasoning output disabled. Paraphrases are generated from the cached question alone.

Claim representation and lexical acceptance.

The monthly replay retains value snaks of both normal and preferred rank, excluding deprecated statements. Preferred statements do not suppress normal statements. Keys encode entity identifiers, quantity amounts, timestamps truncated to eleven characters, or text literals. Qualifiers, references, and other value attributes are omitted.

Base construction and fallback require at least one supplied value to match the answer after removing commas and whitespace and ignoring case. A value text beginning with four digits may also match by that prefix; this fallback is not restricted to typed dates. Assertion extraction applies the lexical rule to individual values.

Every installed asserted set is therefore nonempty. The resulting set is not a semantic decomposition and can include negated or historical mentions.

Similarity screening.

Embeddings use 1,024-dimensional `bge-m3`. For ordered genuine paraphrase similarities,

$$k = \max\{1, \lfloor 0.05 \times 154 \rfloor\} = 7, \qquad \tau_0 = s_{(k)} = 0.8363771188718344.$$

Decisions use rounded threshold $\tau = 0.8364$ and accept on $\geq$, yielding 147 hits and seven misses with no ties. Rounding excludes τ_0 itself. A linearly interpolated 5% quantile gives 146 hits and is not used.

Unified retrieval over all 154×154 query–question pairs ranks every intended entry first and reproduces the seven misses. The minimum genuine-over-impostor margin is 0.071. Entity-grouped five-fold calibration, with cached entries still available for retrieval, yields 144/154 held-out hits and no misrouting. Thresholds range from 0.8384 to 0.8706 (seed 42). This is within-panel calibration, not unseen-cache evaluation.

Comparator implementation.

Table 1 defines the four reuse contracts. Page equality compares wikitext; entity equality compares full claim JSON; property equality compares the projected claim-value set. Value containment checks asserted strings in target-page text, matching dates by year and quantities by number.

The LLM judge uses the base generator with a one-word YES/NO verdict from the cached question, answer, and current fallback evidence. Only YES permits reuse; missing evidence or parse failure declines. It does not assess the incoming paraphrase. An exact validator using the same asserted set and eligibility predicates is decision-equivalent to the asserted-claims gate.

B.3 Answer labels and adjudication

Two non-author annotators independently assess all 154 entries on base correctness (A1), sentence support (A2), paraphrase fidelity (A3), target correctness (A4), and temporal attribution (A5). The sheet contains two-period evidence and survival marks but no gate outputs or staleness labels. Staleness is nevertheless inferable from the evidence. Annotators do not consult source pages or snapshot files.

Agreement is $741/770 = 96.2\%$. A4 agreement is 95.5% ($\kappa = 0.86$); A5 agreement is 94.8% ($\kappa = 0.91$); A3 agreement is 98.1%. The first annotator adjudicates 29 disagreements. The adjudicated labels identify nine temporally decayed entries and three meaning-drifted requests. Using the second annotator's raw classification instead identifies thirteen temporal-decay or incoming-query-error cases; none is cache-served by the four contracts.

The 51 accepted one-shot regenerations are independently labeled for correctness and sentence coverage, with 92.2% agreement on each. Adjudicated correctness is 43 correct and eight incomplete. Coverage is a separate outcome.

Primary multi-tick endpoint assessment additionally labels 175 accepted answers from the merged store: 174 used by core erasure-free policies and one used only by exploratory equality configurations. Agreement is 99.4% ($\kappa = 0.977$); the first annotator adjudicates the single disagreement. These labels and the 51 one-shot labels are disjoint. Labels join to actual answer text, request, and origin tick and assess correctness at the endpoint, including earlier write-backs. Unlabeled fault-run outputs remain unassessed. Raw sheets, adjudications, and parsed labels are retained under `annotation/`.

Appendix C Executed Replay and Protection Protocol

C.1 Generation archive and temporal state updates

The one-shot comparison checks a fixed base cache. Its strict-policy fallback archive covers the relaxed policies' non-serve sets, allowing all policies to share one fallback realization per request.

Multi-tick policies maintain separate caches. Accepted archived outputs receive response-conditioned records constructed from their issuance snapshot and are written back when enabled. The static control delivers fallback outputs without write-back. Regeneration uses the cached question; changed-intent generation is not evaluated.

A logical generation key fixes the request, origin tick, evidence, prompt, model configuration, and acceptance rule. Policies reaching the same key share its archived outcome. Every reachable key must resolve to a model output or explicit abstention; missing coverage is an execution error. Headline runs never substitute template answers.

Archive accounting.

The primary merged store contains 411 records: 370 accepted outputs, 40 no-evidence outcomes, and one acceptance failure. The one-shot store contains 65 records, including 51 accepted outputs. The two accepted-output sets are disjoint. Blind refresh consumes all $370 + 51 = 421$ accepted keys, producing 421 accepted-regeneration events. An event counts an accepted delivered fallback, not an API attempt; several policies may consume the same archived output.

Each generator uses 174 merged-store keys for core erasure-free endpoint delivery. Their sets share 173 keys and have union size 175. One additional primary-generator key is assessed for exploratory equality configurations. Other fault-reachable outputs without human labels remain unassessed.

Acceptance and retries.

Monthly generation uses the lexical rule in Appendix B.2. After an acceptance failure, one additional generation is requested with an appended instruction to name an evidence value. A second failure produces abstention. No usable evidence produces abstention without generation. Transport retries are separate from this acceptance retry.

Second-generator execution.

Regeneration prompts are reissued to `gpt-5-mini`, retaining the primary base cache, requests, screening threshold, evidence states, acceptance rule, and retry limit. Primary requests disable reasoning output; replacement requests do not set reasoning effort. The comparison therefore changes the requested model and potentially its backend configuration together.

New accepted answers receive re-extracted asserted sets and records constructed from their issuance snapshot. All five erasure-free trajectories are re-executed with write-back and archive-coverage assertions. The resulting trajectory map, not substitution into the primary map, defines endpoint delivery. A label is inherited only when normalized answer text, evaluated request, and target assessment context match; other assessed answers are independently labeled. Reissuing fault-reachable prompts does not replicate the fault sweep.

C.2 Placement, failures, and paired estimands

The placement universe contains 720 payload objects referenced by the 147 base similarity hits, totaling 65,508 logical bytes. Claim payloads contain serialized value keys; sentence payloads contain extracted text. Storage keys can be shared across context-specific premise records, whose eligibility is checked separately.

Objects are ranked by certificate-reference frequency, with identifier-order tie breaking. A greedy scan adds each object that fits the remaining budget. The 90% cap selects 58,954 bytes.

Placement is fixed across ticks. Write-back payload keys outside this universe remain unprotected. The random arm shuffles the payload-object universe at the start of each realization and stores each item that fits the remaining 58,954-byte budget. The resulting placement remains fixed across ticks. This executed shuffled-fill comparator differs from both independent Bernoulli placement and the rejection-sampled static comparator in Supplementary Section S6.

The ideal read-any reference makes every eligible required payload readable, including write-back premises. It retains eligibility checking but removes read loss; it is not complete placement with two fallible routes.

Fault coupling.

Fault keys are (seed, tick, request, premise, read route). Arms share draws for identical keys; source and backup routes fail independently at rate p. Repeated references within a certificate count once, whereas different requests may have different faults for the same premise. Each cell uses 200 realizations.

Source-route availability is evaluated from the snapshot registry and injected read faults; selected backup payloads are read from SQLite. Faults affect reuse-path checks only. Fallback uses archived current-snapshot evidence without these faults. Paired transitions compare policy-specific write-back trajectories, not protection toggles at identical cache states.

Storage accounting.

Bytes count unique payloads, not certificate-reference sums or physical database size. The frozen panel uses the same rule: 13,377 protected bytes from a 14,887-byte universe of 162 items.

C.3 Measurement and uncertainty

Local timings use `perf_counter_ns` on one Apple M3 Pro with materialized evidence. Serve-path stages report medians over 3,000 randomized hits; payload-chain operations report means. Query embedding, remote acquisition, and complete upkeep are excluded. Snapshot loading occurs once per update tick.

Bootstrap estimands.

Similarity-hit rate intervals use 10,000 entity-cluster resamples of the 136 entities represented among the 147 hits (seed 42). Paired endpoint-quality intervals use 10,000 resamples of the 143 entities in the full 154-request panel (seed 20261017). For policy pair A, B, correct-delivery indicators Y_r^A, Y_r^B, and cluster multiplicities $m_{c(r)}^{(b)}$, the fixed-panel count contrast is

$$\Delta_b = 154 \, \frac{\sum_r m_{c(r)}^{(b)} (Y_r^A - Y_r^B)}{\sum_r m_{c(r)}^{(b)}}.$$

This rescales each draw to 154 requests. Intervals condition on the panel and archived outputs, not repeated model generation.

The frozen-check interval instead resamples 100 paired fault realizations using 2,000 bootstrap draws. All bootstrap intervals use the empirical 2.5th and 97.5th percentiles.

Frozen mechanism check.

A `Random(20261017)` shuffle orders non-design candidates with pages at every execution state. The first 50 accepted pairs are retained after 181 candidates. The fixed rule protects 154/162 items. No natural ineligibility case occurs; forced-loss and eligibility-flip tests exercise the guards. Fresh entries and template fallback limit the check to restoration mechanics.

Appendix D Supplementary Results and Artifact Guide

Online Resource 1 contains extended protocols and results. Table D1 provides a section map; its Section S10 gives the file-level inventory. Table D2 identifies the core monthly execution outputs. Primary endpoint counts use the label-aware real-driven execution. The earlier `tableb_endpoint_map.json` records archive construction and does not define the reported endpoint routing.

Table D1 Guide to Online Resource 1.

Section	Material
S1	Controlled availability, retrieval, trace-policy accounting, and small-DAG checks
S2	Numerical coding, shared-root, and alternative-family references
S3–S5	Snapshot dynamics, monthly replay details, generator sensitivity, and exploratory alternatives
S6–S7	Static protection models, executed diagnostics, certificate handoff, and local timings
S8	Daily service-demand scenario and access-cost conventions
S9	Daily panel, routing protocol, and projected-error sensitivity
S10	Experiment and output inventory

Table D2 Core monthly execution outputs under `experiments/14_snapshot_serving_replay/`.

Result	Output
Endpoint execution and labels	`real_driven_loop.json`, `tableb_labels.json`, `regen_labels.json`
Executed trajectories and faults	`real_driven_loop.json`, `real_driven_sweep.json`
Generation coverage	`tableb_regeneration.json`
Residual and paired accounting	`residual_decomposition.json`, `paired_transitions_real.json`
Frozen mechanism check	`frozen_panel_generation.json`, `frozen_panel_check.json`

References

[1] Bang, F.: GPTCache: an open-source semantic cache for LLM applications enabling faster answers and cost savings. In: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023), pp. 212–218 (2023)

[2] Schroeder, L.G., Desai, A., Cuadron, A., *et al.*: vCache: verified semantic prompt caching. In: International Conference on Learning Representations (2026)

[3] Singh, A.K., Wang, H., Attaluri, L.N.S., *et al.*: Asynchronous verified semantic caching for tiered LLM architectures. In: Proceedings of the Sixth European Workshop on Machine Learning and Systems, pp. 192–199 (2026)

[4] Gill, W., Elidrisi, M., Kalapatapu, P., *et al.*: MeanCache: user-centric semantic caching for LLM web services. In: Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 1298–1310 (2025). https://doi.org/10.1109/IPDPS64566.2025.00117

[5] Xu, J.: Partial path interception in derivation DAGs: exact reliability laws and a sharp complexity dichotomy. Version 3; earlier versions appeared under the title "Proof-Valid Caching under Premise Erasures" (2026)

[6] vCache Project: SemCache benchmark datasets and evaluation code. GitHub Repository. https://github.com/vcache-project/vCache. Accessed August 2026 (2026)

[7] Jin, C., Zhang, Z., Jiang, X., *et al.*: RAGCache: efficient knowledge caching for

retrieval-augmented generation. ACM Trans Comput Syst **44**(1), 1–27 (2025) https://doi.org/10.1145/3768628

[8] Yao, J., Li, H., Liu, Y., *et al.*: CacheBlend: fast large language model serving for RAG with cached knowledge fusion. In: Proceedings of the Twentieth European Conference on Computer Systems, pp. 94–109 (2025)

[9] Kwon, W., Li, Z., Zhuang, S., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp. 611–626 (2023)

[10] Zhong, Y., Liu, S., Chen, J., *et al.*: DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In: Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation, pp. 193–210 (2024)

[11] Zheng, L., Yin, L., Xie, Z., *et al.*: SGLang: efficient execution of structured language model programs. In: Advances in Neural Information Processing Systems, vol. 37, pp. 62557–62583 (2024). https://doi.org/10.52202/079017-2000

[12] Es, S., James, J., Espinosa Anke, L., *et al.*: RAGAs: automated evaluation of retrieval augmented generation. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158 (2024)

[13] Saad-Falcon, J., Khattab, O., Potts, C., *et al.*: ARES: an automated evaluation framework for retrieval-augmented generation systems. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 338–354 (2024)

[14] Ru, D., Qiu, L., Hu, X., *et al.*: RAGChecker: a fine-grained framework for diagnosing retrieval-augmented generation. In: Advances in Neural Information Processing Systems, vol. 37, pp. 21999–22027 (2024)

[15] Buneman, P., Khanna, S., Tan, W.C.: Why and where: a characterization of data provenance. In: Proceedings of the 8th International Conference on Database Theory, pp. 316–330 (2001)

[16] Cheney, J., Chiticariu, L., Tan, W.C.: Provenance in databases: why, how, and where. Found Trends Databases **1**, 379–474 (2009)

[17] Kim, J., Pasini, S., Tonella, P.: When knowledge changes: metamorphic testing of RAG systems with mutations. In: Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE) (2026). https://doi.org/10.1145/3832783.3837543

[18] Jang, J., Ye, S., Lee, C., *et al.*: TemporalWiki: a lifelong benchmark for training and evaluating ever-evolving language models. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, pp. 6237–6250 (2022)

[19] Ammar Khodja, H., Béchet, F., Brabant, Q., *et al.*: WikiFactDiff: a large, realistic, and temporally adaptable dataset for atomic factual knowledge update in causal language models. In: Proceedings of the Joint International Conference on Computational Linguistics, Language Resources and Evaluation, pp. 17614–17624 (2024)

[20] Kasai, J., Sakaguchi, K., Takahashi, Y., *et al.*: RealTimeQA: what's the answer right now? In: Advances in Neural Information Processing Systems (2023)

[21] Vu, T., Iyyer, M., Wang, X., *et al.*: FreshLLMs: refreshing large language models with search engine augmentation. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 13697–13720 (2024)

[22] Cheng, J., Dou, Z.: DailyQA: a benchmark to evaluate web retrieval augmented LLMs based on capturing real-world changes. arXiv:2505.17162 (2025)

[23] Shah, S.H.: Grounded cache routing for retrieval-augmented generation: when is it safe to reuse an answer? arXiv:2605.27494v1 (2026)

[24] Zeng, L., Jin, Y.: FinCacheServe: dependency-consistent answer reuse for cost-efficient RAG serving over mutable enterprise documents. arXiv:2607.26076v1 (2026)

[25] Yoon, Y., et al.: Risk-constrained freshness-aware semantic caching for open-web retrieval-augmented LLMs. arXiv:2607.04281v1 (2026)

[26] Maddah-Ali, M.A., Niesen, U.: Fundamental limits of caching. IEEE Trans Inf Theory **60**, 2856–2867 (2014)

[27] Yu, Q., Maddah-Ali, M.A., Avestimehr, A.S.: The exact rate-memory tradeoff for caching with uncoded prefetching. IEEE Trans Inf Theory **64**(2), 1281–1296 (2018)

[28] Reed, I.S., Solomon, G.: Polynomial codes over certain finite fields. J Soc Ind Appl Math **8**, 300–304 (1960)

[29] Singleton, R.C.: Maximum distance q-nary codes. IEEE Trans Inf Theory **10**, 116–118 (1964)

[30] MacWilliams, F.J., Sloane, N.J.A.: The Theory of Error-Correcting Codes. North-Holland, Amsterdam (1977)

[31] Plank, J.S., Blaum, M., Hafner, J.L.: SD codes: erasure codes designed for how storage systems really fail. In: Proceedings of the 11th USENIX Conference on

File and Storage Technologies, pp. 95–104 (2013)

[32] Yang, Z.L., Qi, P., Zhang, S.Z., *et al.*: HotpotQA: a dataset for diverse, explainable multi-hop question answering. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing, pp. 2369–2380 (2018)

[33] Trivedi, H., Balasubramanian, N., Khot, T., *et al.*: MuSiQue: multihop questions via single-hop question composition. Trans Assoc Comput Linguist **10**, 539–554 (2022)

[34] Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, pp. 3982–3992 (2019)

[35] Wikimedia Foundation: Wikimedia database backup dumps: enwiki and wikidatawiki snapshots, 2026-04-01 to 2026-08-01. https://dumps.wikimedia.org/. Accessed August 2026 (2026)